Winter AI Study
Week 2: Basic Deep Learning

발표자 조현진

Basic Deep Learning

이번 주차 스터디의 흐름

MLP

CNN

Recap: 머신러닝 vs 딥러닝

머신러닝

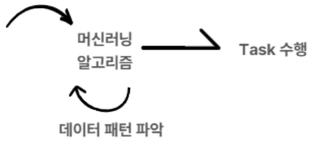
머신러닝 vs 딥러닝 딥러닝이 아닌 머신러닝: 정형 데이터의 패턴을 파악

정형 데이터: 행과 열에 인간이 규정한 의미가 부여되어 있는 데이터

	Gene 1	Gene 2	Gene 3
Cell 1	0.5	0.7	0.2
Cell 2	0.4	0.2	0.1
Cell 3	0.7	0.9	0.6

Gene 1, 2, 3는 Cell을 표현하는 feature

인간이 직접 feature를 설정해놓아야 함 (딥러닝과의 핵심 차이)



딥러닝

머신러닝 vs 딥러닝 딥러닝: 비정형 데이터의 패턴도 파악 가능 (Feature Learning 기법)

Feature Learning?

모델이 데이터 내부에 내재되어 있는 feature들을 직접 뽑아내 학습 (!= 머신러닝은 feature를 직접 인간이 설계)

딥러닝 적용 분야 두 대표

Computer Vision (CV)
컴퓨터가 이미지를 이해하는 방식을 탐구하는 분야

Natural Language Processing (NLP, 자연어처리)
컴퓨터가 자연어를 이해하는 방식을 탐구하는 분야

딥러닝 Feature

물론 정형데이터도 벡터, 행렬 등의 표현을 쓸 수는 있음 (모양만 따지면)
각 차원 자리에 존재하는 숫자가 어떠한 의미를 지니고 있는지는 비정형 데이터에서는 모름

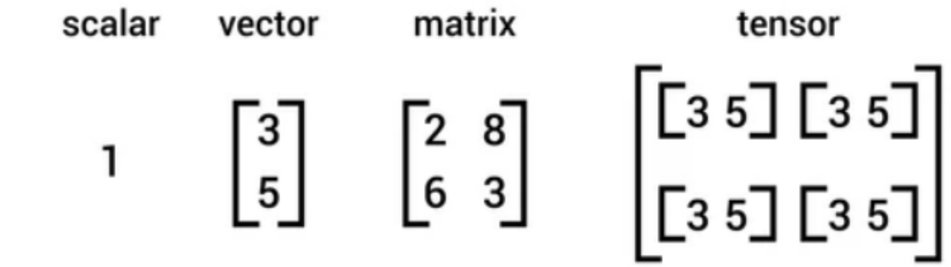
머신러닝

각 feature가 어떤 의미인지 사람이 이미 설계(정형 데이터)

	Feature 1	Feature 2	Feature 3
Sample 1			
Sample 2			
Sample 3			

딥러닝

scalar vector matrix tensor



딥러닝 모델은 여러 computation layer가 쌓여 있는 형태를 가지고 각 layer에 대해서 input도 이런 형태, 중간 값(은닉층 값)도 이런 형태, output도 이런 형태로 표현해야함

따라서 딥러닝을 잘 이해하려면 basic한 **선형대수** 개념정도는 알고 있어야 함

Basic Linear Algebra

벡터(Vector)?

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

1차원 형태의 숫자 모음

상황에 따라 다르게 해석

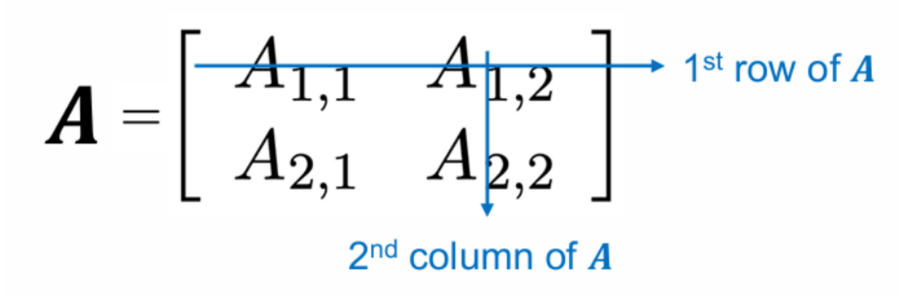
공통적으로 해당 x를 n차원 좌표공간의 점이라고 생각해도 됨

$$\boldsymbol{x} \in \mathbb{R}^n$$

벡터의 차원 형태를 표현하는 방식 (익숙해지시면 좋습니다)

Basic Linear Algebra

행렬(Matrix)?



2차원 형태의 숫자 모음

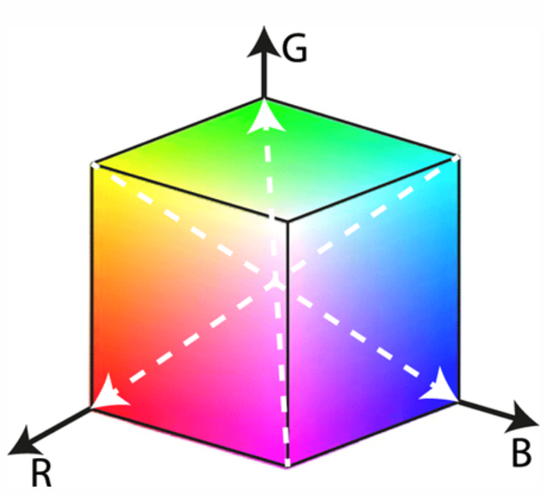
상황에 따라 다르게 해석

$$\boldsymbol{A} \in \mathbb{R}^{m \times n}$$

행렬의 차원 형태를 표현하는 방식 (익숙해지시면 좋습니다)

Basic Linear Algebra

텐서(Tensor)?



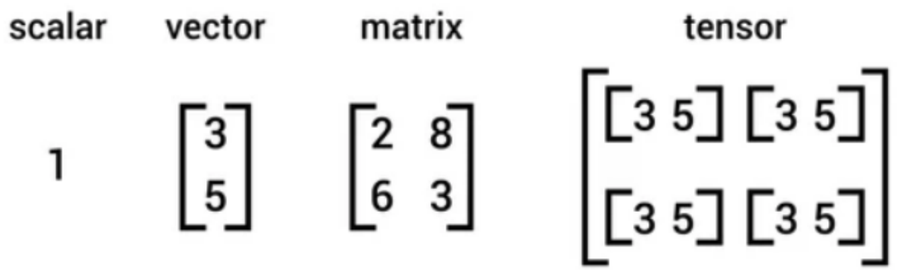
3차원 이상 숫자 모음

상황에 따라 다르게 해석

텐서의 예 $x \in \mathbb{N}^{3\times128\times128}$

텐서의 차원 형태를 표현하는 방식 (익숙해지시면 좋습니다)

Basic Linear Algebra



딥러닝의 input부터 중간 값(은닉층 값), output까지 모두 이 형태의 값들을 사용

Basic Linear Algebra

선형 연산?

Regression(회귀) Task로 공부하는 머신러닝 (딥러닝 x)

Regression: 수치 변수 예측 (즉, 값을 예측)

회귀 머신러닝 알고리즘 3: Multiple Linear Regression

$$y = w1*x1 + w2*x2 + w3*x3 + ...$$

	x1	x2	x3
Sample 1			
Sample 2			
Sample 3			

w1 학습: 입력 feature x1에 가중치를 얼마나 둬야 하나 학습
w2 학습: 입력 feature x2에 가중치를 얼마나 둬야 하나 학습
....

대부분의 정형 데이터는 입력 feature가 1개로만 구성되어 있지는 않음

물론 수학적으로 엄밀한 정의가 있지만...

대충 y = (가중치) * (입력 값)

딥러닝의 많은 layer가 이런 형태의 연산을 취함

Basic Linear Algebra

선형 연산?

예를 들어 output 벡터 $y = [y1, y2, y3] \in R^3$ 가 있고, input 벡터 $x = [x1, x2, x3] \in R^3$ 가 있다고 했을 때,

$y1 = W1*x1 + W2*x2 + W3*x3$
$y2 = W4*x1 + W5*x2 + W6*x3$
$y3 = W7*x1 + W8*x2 + W9*x3$

이런 식으로 y를 계산하도록 모델을 설계했다고 해보자

→ Matrix와 vector 표현을 사용하면 하나의 계산식으로 표현 가능함

Basic Linear Algebra

선형 연산?

예를 들어 output 벡터 $y = [y1, y2, y3] \in R^3$ 가 있고, input 벡터 $x = [x1, x2, x3] \in R^3$ 가 있다고 했을 때,

y1 = W1*x1 + W2*x2 + W3*x3
y2 = W4*x1 + W5*x2 + W6*x3
y3 = W7*x1 + W8*x2 + W9*x3

이런 식으로 y를 계산하도록 모델을 설계했다고 해보자

W1	W2	W3	*	x1	=	y1 = W1*x1 + W2*x2 + W3*x3
W4	W5	W6		x2		y2 = W4*x1 + W5*x2 + W6*x3
W7	W8	W9		x3		y3 = W7*x1 + W8*x2 + W9*x3

Basic Linear Algebra

행렬 곱

W1	W2	W3
W4	W5	W6
W7	W8	W9

*

x1	x2	x3
x4	x5	x6
x7	x8	x9

=

W1*x1 + W2*x4 + W3*x7		

Basic Linear Algebra

행렬 곱

W1	W2	W3
W4	W5	W6
W7	W8	W9

*

x1	x2	x3
x4	x5	x6
x7	x8	x9

=

y1	W1*x2 + W2*x5 + W3*x8	

Basic Linear Algebra

행렬 곱

W1	W2	W3
W4	W5	W6
W7	W8	W9

*

x1	x2	x3
x4	x5	x6
x7	x8	x9

=

y1	y2	y3
W4*x1 + W5*x4 + W6*x7		

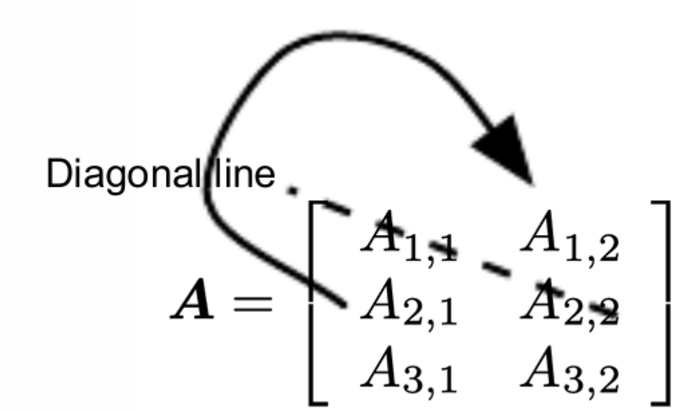
Basic Linear Algebra

행렬 곱 차원 계산

(m x n 행렬) * (n x d 행렬) = (m x d 행렬)

Basic Linear Algebra

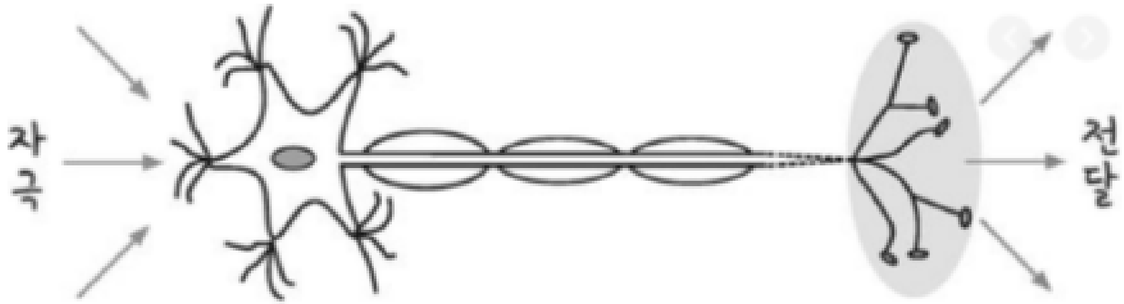
Transpose 행과 열을 바꿈



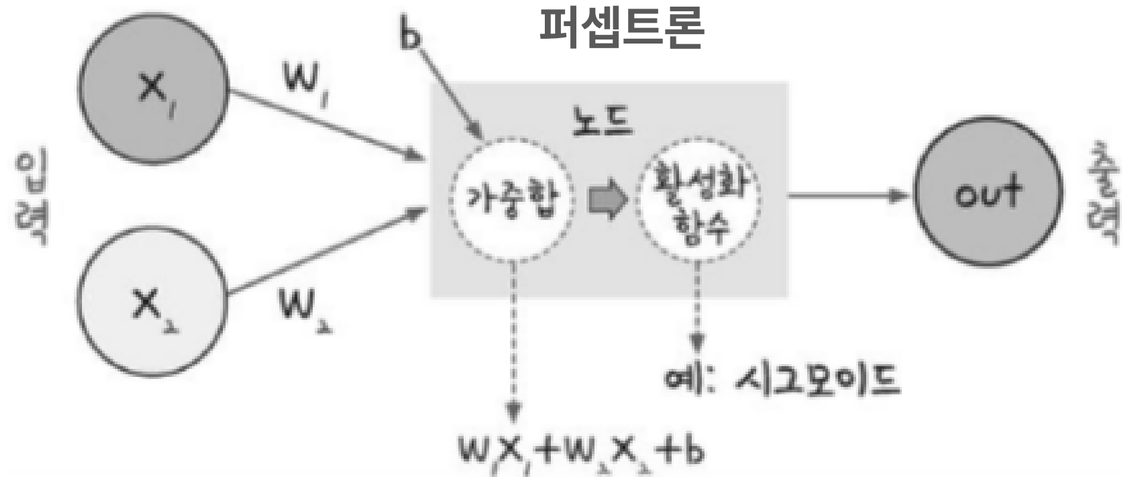
$$\boldsymbol{A} = \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \\ A_{3,1} & A_{3,2} \end{bmatrix} \Rightarrow \boldsymbol{A}^\top = \begin{bmatrix} A_{1,1} & A_{2,1} & A_{3,1} \\ A_{1,2} & A_{2,2} & A_{3,2} \end{bmatrix}$$

3 x 2 행렬 2 x 3 행렬

Perceptron



뉴런을 모방한 딥러닝의 가장 기본 단위



입력이라는 자극이 들어오면

가중합(선형 연산)이라는 신호 통합이 이루어지고

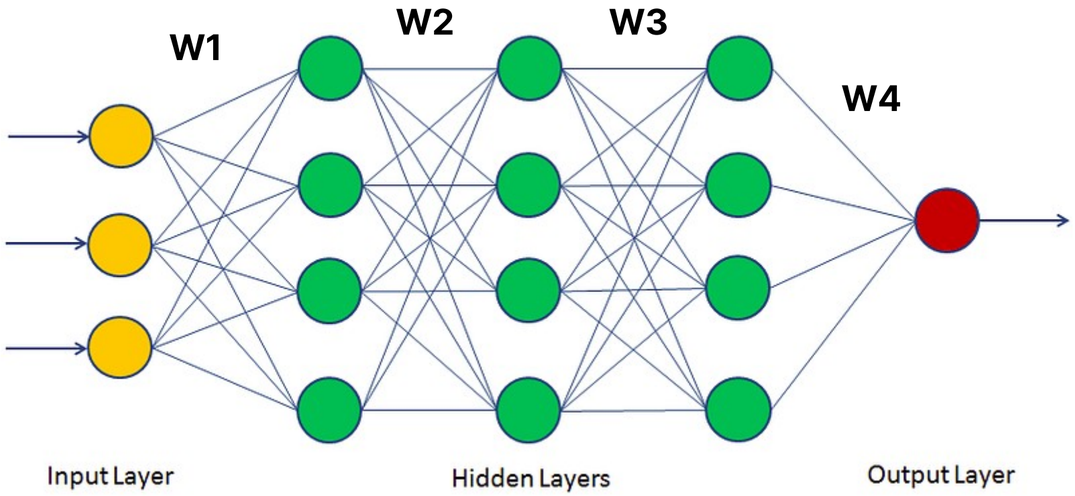
활성화함수를 거쳐 activated output을 낸다.

뒤 슬라이드 설명 예정

Multi-Layer Perceptron (MLP)

각 layer의 노드들이 모두 다 연결이 되어 있으면 fully-connect layer(FC layer)라고 부름

hidden layer는 중간에 나오는 값들을 의미 (은닉층 값이라고 부름)



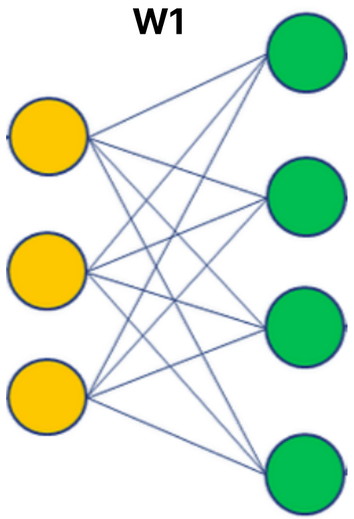
퍼셉트론을 여러 층 쌓아 만든 기초적인 딥러닝 모델

주로 정보가 혼합된 벡터(input)를 모델에 이해시키고자 할 때 사용

각 layer는 모델이 아닌 데이터 값임 ex) 한 층에 노드가 4개면 4 x 1 벡터

실제 모델을 표현하는 파라미터 W는 엣지에 그 의미가 담겨져 있음

Multi-Layer Perceptron (MLP)



$$W1 \in R^{? \times ?}$$

우리는 이 perceptron을 구현하기 위해 파라미터 W1을 몇 차원 행렬로 둬야할까?

perceptron 식

$$h1 = W1 * x$$

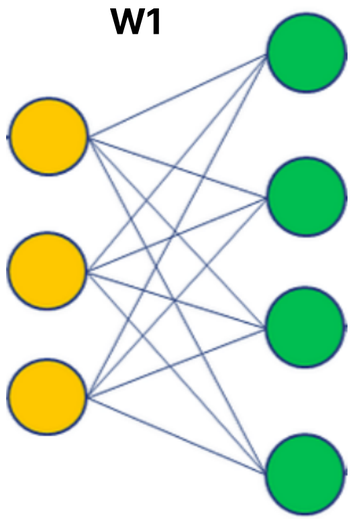
4 x 1 ? x ? 3 x 1

Recap
(m x n 행렬) * (n x d 행렬) = (m x d 행렬)

$$? \times ? = 4 \times 3$$

아~ 이 모델의 파라미터 개수는 12개이구나

Multi-Layer Perceptron (MLP)



$x \in R^3$ $h1 \in R^4$

W11	W12	W13
W14	W15	W16
W17	W18	W19
W110	W111	W112

W1

*

x1
x2
x3

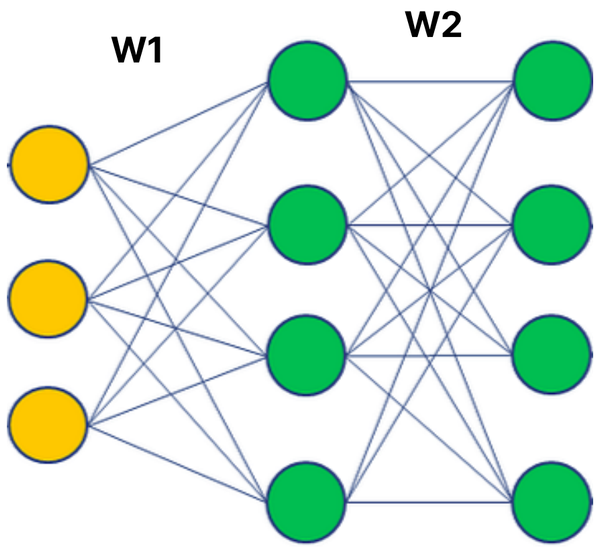
x

=

h11
h12
h13
h14

h1

Multi-Layer Perceptron (MLP)



$x \in R^3$ $h1 \in R^4$ $h2 \in R^4$

W21	W22	W23	W24
W25	W26	W27	W28
W29	W210	W211	W212
W213	W214	W215	W216

W2

*

h11
h12
h13
h14

h1

=

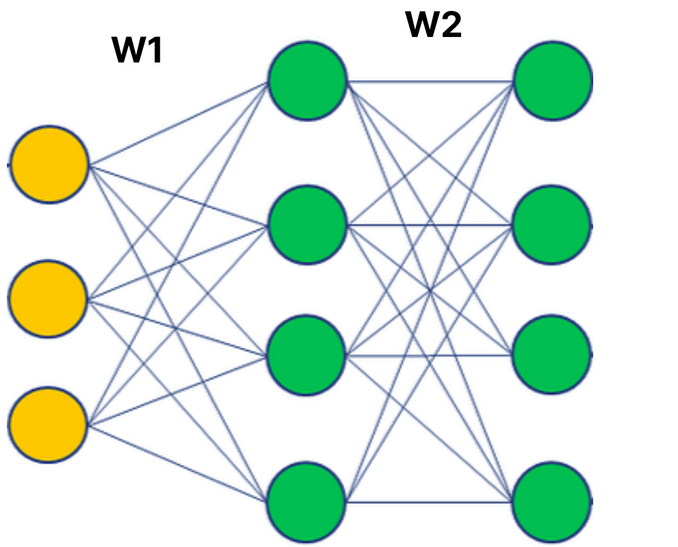
h21
h22
h23
h24

h2

$$h2 = W2 * h1$$

4 x 1 4 x 4 4 x 1

Multi-Layer Perceptron (MLP)



$$h1 = W1 * x$$

$$h2 = W2 * h1$$

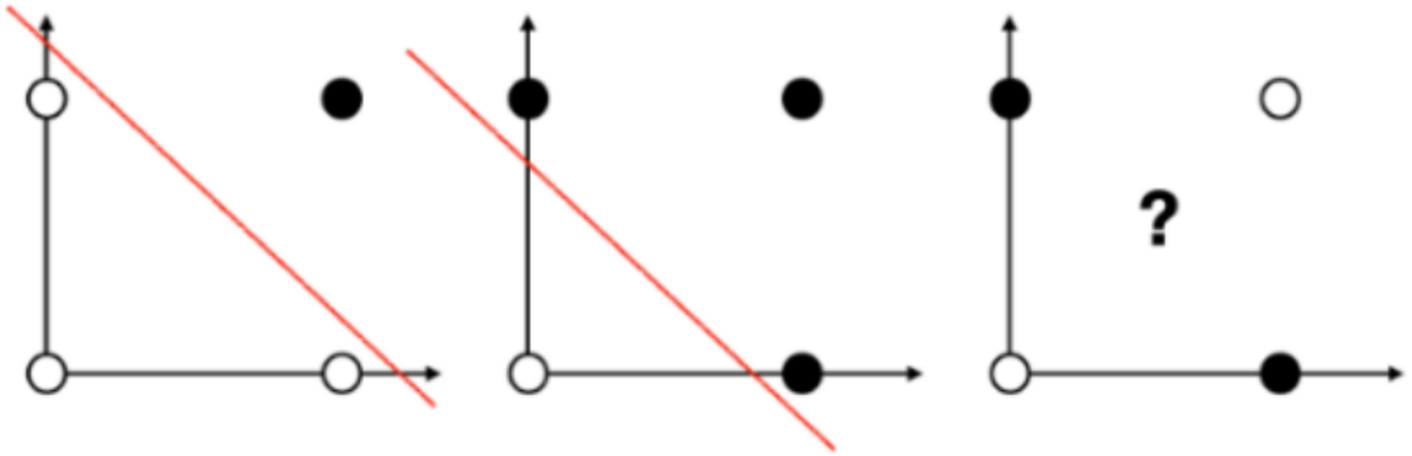
$$h2 = W2 * W1 * x$$

$$= W_{new} * x$$

이러면 층을 하나만 사용하는 것과 다를 바가 없음!

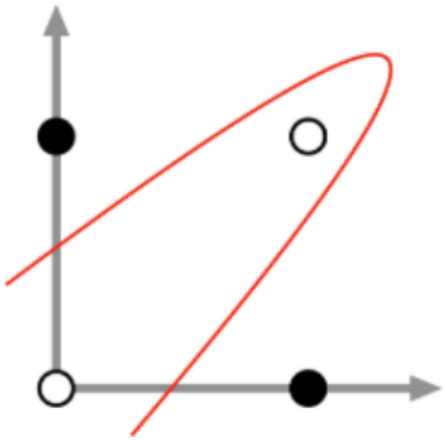
→ Activation function(활성화 함수)이 필요한 이유

Multi-Layer Perceptron (MLP)



첫 번째, 두 번째는 분류 경계선을 선형 연산만으로 나눌 수 있음

세 번째와 같이 복잡한 데이터 분포에 대해서 선형 연산만으로는 분류 경계선을 나눌 수 없음



복잡한 데이터 분포에 대해서는
선형 연산만 할 것이 아닌 비선형 연산 역시 필요

Multi-Layer Perceptron (MLP)

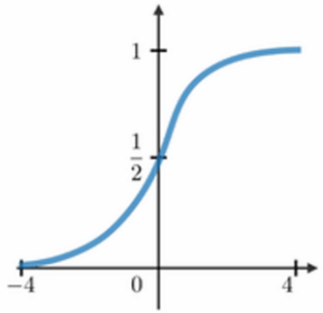
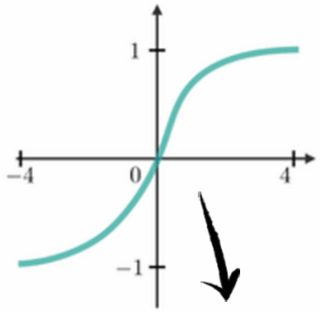
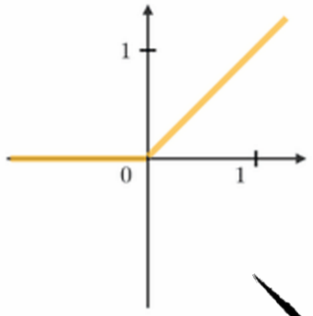
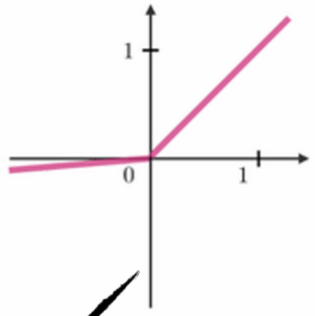
선형 연산으로만 이루어진 layer에 비선형 연산을 추가할 수 있는 방법

→ Activation function 사용

Activation function은 입력 값의 차원을 절대 바꾸지 않음

이진 분류 확률 분포 계산용 (다중 분류면 softmax: 1주차 참고)

Activation function examples:

Sigmoid	Tanh	ReLU	Leaky ReLU
$g(z) = \frac{1}{1+e^{-z}}$	$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$g(z) = \max(0, z)$	$g(z) = \max(\epsilon z, z)$ with $\epsilon \ll 1$
			

RNN & LSTM 등에 사용

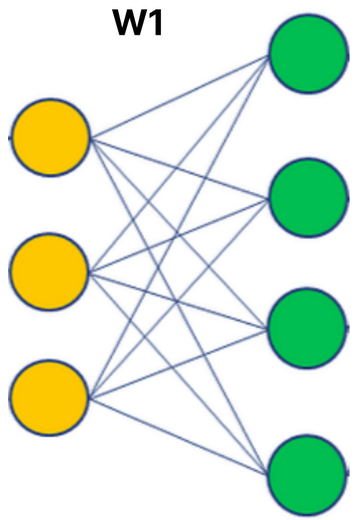
MLP 활성화 함수로 많이 사용

이것말고도 다양...

(활성화 함수를 둬야할 때 어떤 함수를 둘 지 search해보기)

Multi-Layer Perceptron (MLP)

f: activation function



$x \in R^3$ $h1 \in R^4$

$$h1 = W1 * x$$

h1을 바로 다음 layer로 전달하지 말고...

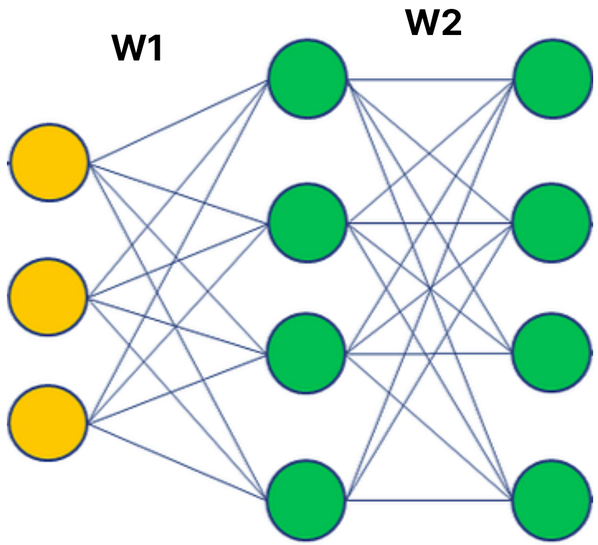
$$\underset{4 \times 1}{a1} = f(\underset{4 \times 1}{h1})$$

Activation 값

이것이 다음 layer로 전달되는 값임

Multi-Layer Perceptron (MLP)

f: activation function



$x \in R^3$ $h1 \in R^4$ $h2 \in R^4$

$$h2 = W2 * a1$$

h2를 바로 다음 layer로 전달하지 말고...

$$\underset{4 \times 1}{a2} = f(h2)$$

4 x 1

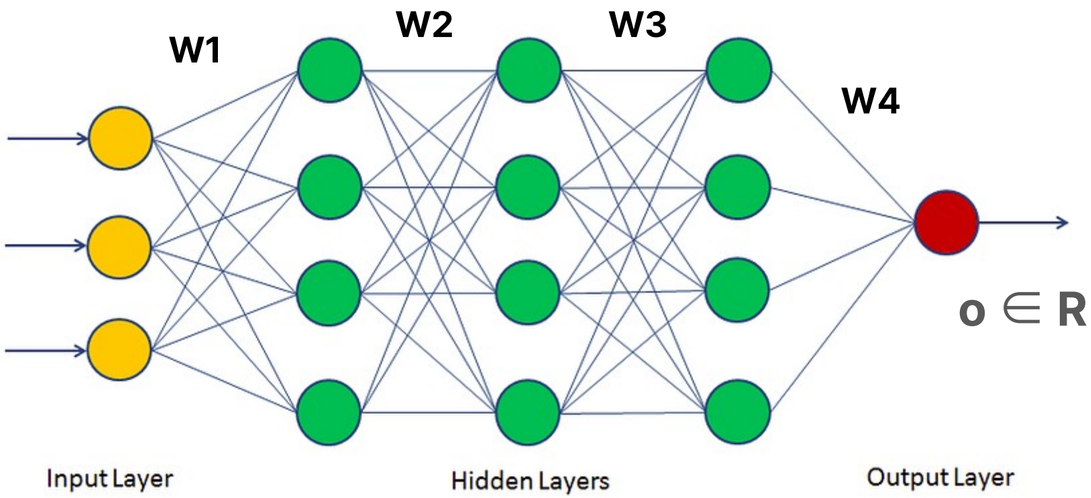
Activation 값

이것이 다음 layer로 전달되는 값임

Multi-Layer Perceptron (MLP): Forward Pass

f: activation function

Obj: objective function (ex. loss)



$o \in R^1$

이것을 최소화하거나 최대화하는 파라미터를 훈련 중에 찾아야 함

Loss면 최소화

Obj(o, 정답지)

$x \in R^3$ $h1 \in R^4$ $h2 \in R^4$ $h3 \in R^4$

$h1 = W1 * x$

$a1 = f(h1)$

$h2 = W2 * a1$

$a2 = f(h2)$

$h3 = W1 * a2$

$a3 = f(h3)$

$h4 = W4 * a3$

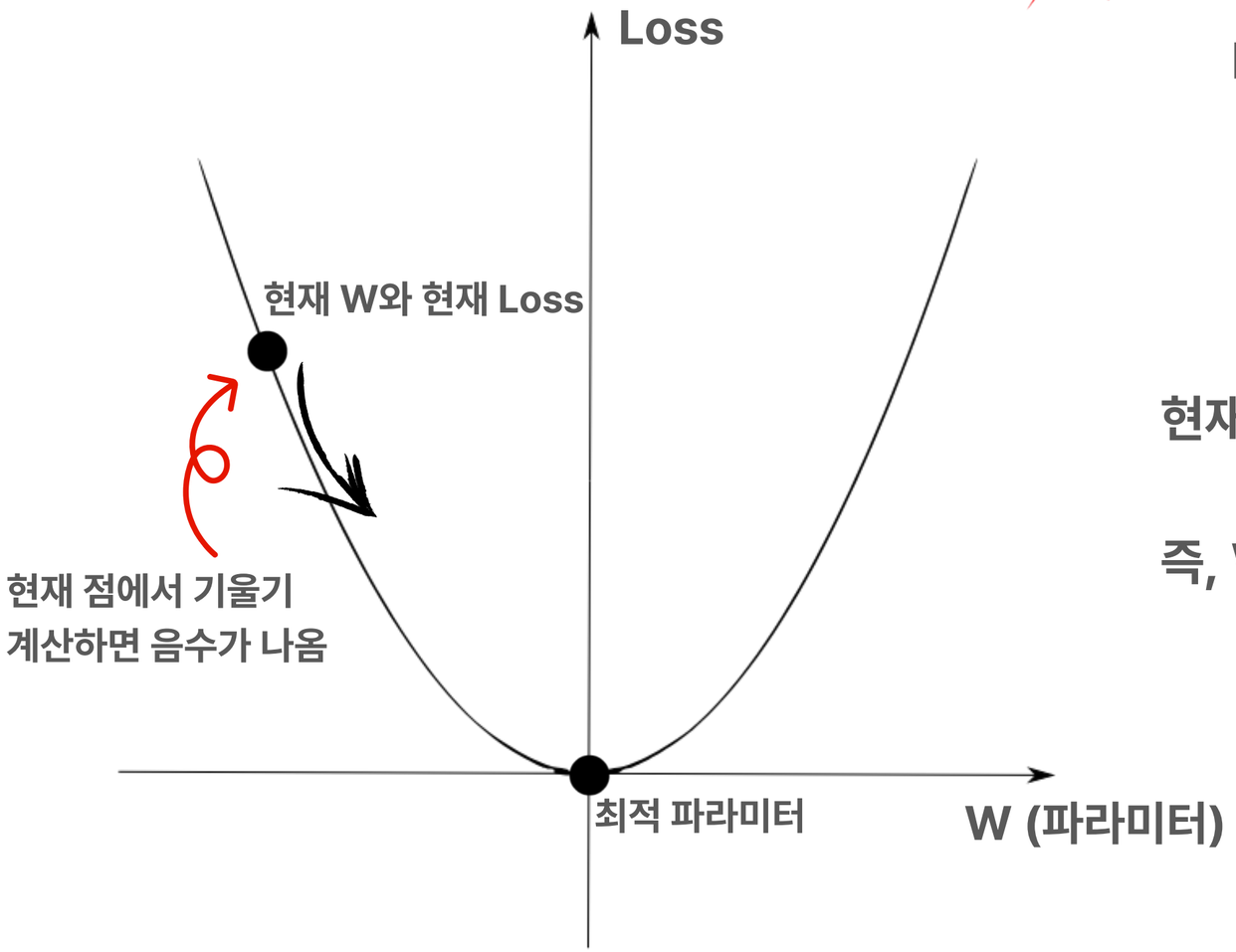
$o = f(h4)$

Activation 값을 연산하고 input에서 output까지 앞에서 뒤로 **전달**: **Forward Pass**라고 부름

Input과 h, ouput의 차원은 설계자 마음

다만, input 차원과 output 차원은 task에 따라 적절한 차원 수가 있을 것이고, h 차원도 관습적인 패턴이 있음 (ex. 256 → 128 → 64 → ...)

Gradient Descent (GD, 경사하강법)

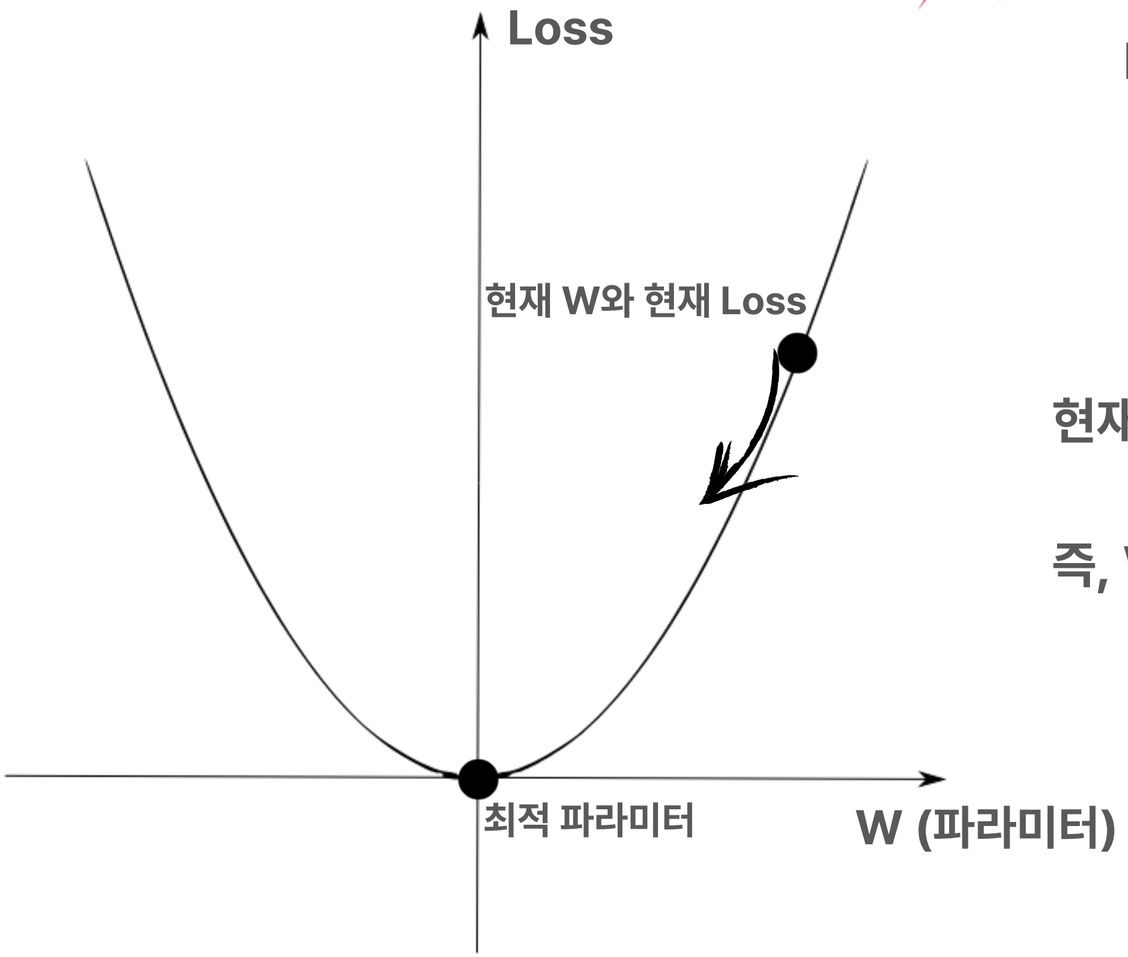


Loss = f(W) (편의상 파라미터 W만 존재하는 모델)

현재 점에서 최적 파라미터로 가려면 오른쪽으로 이동해야함

즉, Wnew = Wcurrent + ? 이렇게 계산되어야 함

Gradient Descent (GD, 경사하강법)

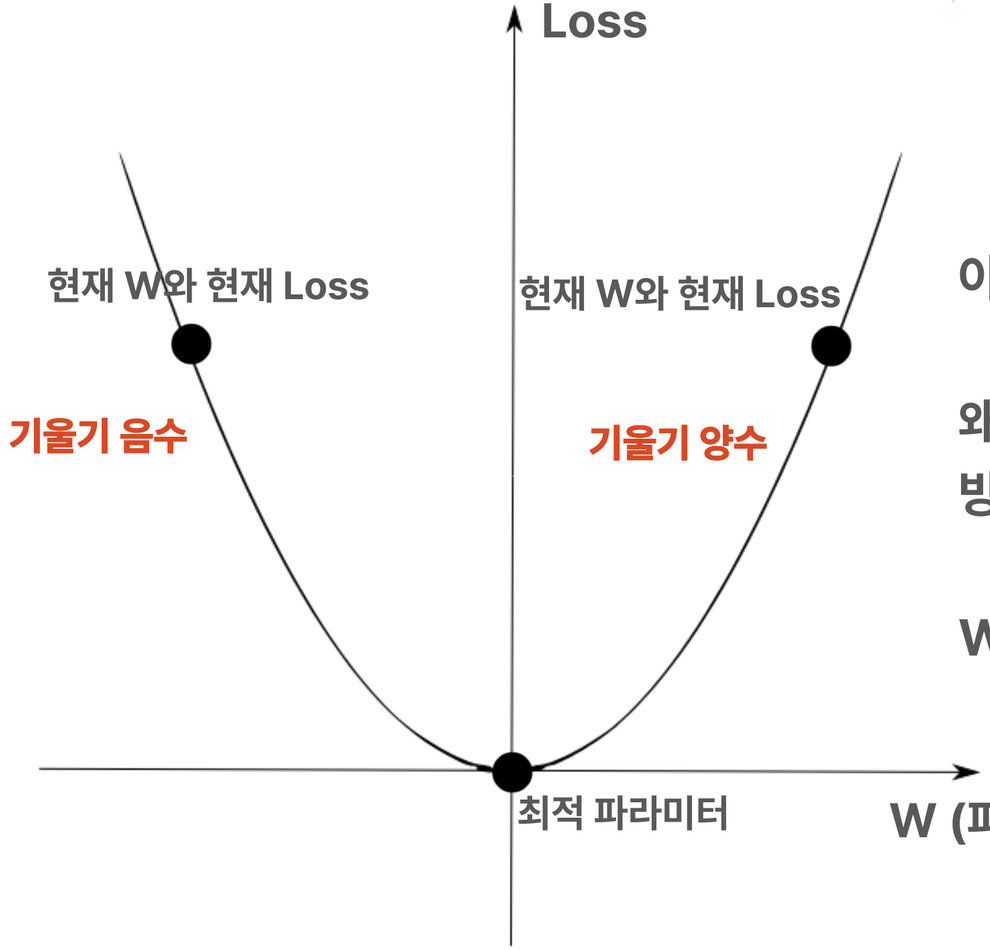


Loss = f(W) (편의상 파라미터 W만 존재하는 모델)

현재 점에서 최적 파라미터로 가려면 왼쪽으로 이동해야함

즉, Wnew = Wcurrent - ? 이렇게 계산되어야 함

Gradient Descent (GD, 경사하강법)



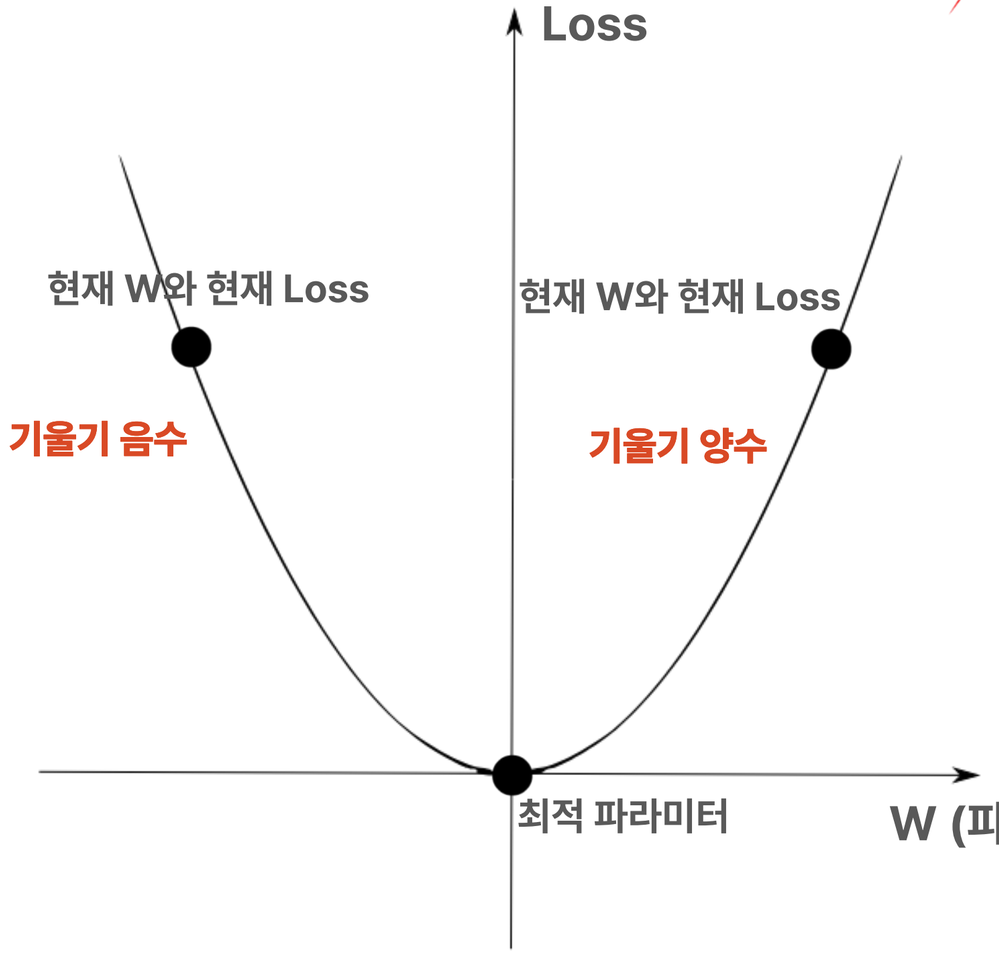
Loss = f(W) (편의상 파라미터 W만 존재하는 모델)

아 이제보니깐 기울기의 부호가 W 최적점을 찾기 위한 방향을 제공해주네

왜 그렇냐고?
방금 case 2개를 일반화 해보면 돼!

Wnew = Wcurrent - gradient

Gradient Descent (GD, 경사하강법)



Loss = f(W) (편의상 파라미터 W만 존재하는 모델)

Wnew = Wcurrent - gradient

gradient의 크기가 너무 크게 계산되면 W의 조정 보폭이 너무 크고

gradient의 크기가 너무 작게 계산되면 W의 조정 보폭이 너무 작게 됨

이를 조절하기 위해 learning rate 상수 값을 도입

Wnew = Wcurrent - (learning_rate) x gradient (GD)

gradient = d(Loss) / d(W)

Full-Batch Gradient Descent (Full-Batch GD)

gradient = d(Loss) / d(W)

Wnew = Wcurrent - (learning_rate) x gradient (GD)

모델 학습 용어 1

Batch란?

User Inference할 때는 보통 샘플 1개를 모델에 인풋으로 넣으면 모델이 그 샘플에 대한 output을 연산함

Training할 때는 샘플 여러 개를 묶어 한 번에 모델에 인풋으로 넣고 각 샘플에 대응하는 output들을 한 번에 연산
Training시, 같이 모델에 들어가는 샘플 묶음을 batch라고 부름
(사실 Inference 때도, 배치 처리를 지원하는 모델이면 한 번에 여러 샘플의 결과를 받아낼 수 있음)

Full-Batch == 데이터셋 내 모든 샘플을 한 묶음(batch 1개)으로 만듦

Full-Batch Gradient Descent (Full-Batch GD)

gradient = d(Loss) / d(W)

Wnew = Wcurrent - (learning_rate) x gradient (GD)

모델 학습 용어 2

Epoch이란?

모델이 데이터셋 내 모든 샘플에 대해 학습을 1번 진행했다고 학습이 끝나는 것이 아님

모델이 데이터셋 내 모든 샘플에 대해 학습 완료하면 1 epoch

모델 학습 시 보통 n epoch을 돌림

Full-Batch Gradient Descent (Full-Batch GD)

gradient = d(Loss) / d(W)

Wnew = Wcurrent - (learning_rate) x gradient (GD)

한 batch 내 각 샘플에 대해 계산된 기울기들의 평균

샘플 1개당 기울기

$$\frac{\partial L(w_t)}{\partial w_t} = \frac{1}{B}\sum_{k=1}^{B}\frac{\partial L_k(w_t)}{\partial w_t}$$

한 배치 내 모든 샘플에 대한
기울기를 평균낸
평균 기울기를 GD식에 대입

B: Batch size
(full-batch의 경우 데이터셋 size와 동일)

$$w_{t+1} = w_t - \eta\frac{\partial L(w_t)}{\partial w_t}$$

Full-Batch GD 식 (B = dataset size 일 때)

Stochastic Gradient Descent (SGD, 확률적 경사하강법)

gradient = d(Loss) / d(W)

Wnew = Wcurrent - (learning_rate) x gradient (GD)

한 batch 내 각 샘플에 대해 계산된 기울기들의 평균

샘플 1개당 기울기

$$\frac{\partial L(w_t)}{\partial w_t} = \frac{1}{B}\sum_{k=1}^{B}\frac{\partial L_k(w_t)}{\partial w_t}$$

B: Batch size

(SGD의 경우 B = 1)

B = 1이므로, 사실상
평균 값이 아닌 샘플 1개당
기울기로 파라미터 업데이트

$$w_{t+1} = w_t - \eta\frac{\partial L(w_t)}{\partial w_t}$$

SGD 식 (B = 1 일 때)

Mini-Batch Gradient Descent (미니 배치 경사하강법)

gradient = d(Loss) / d(W)

Wnew = Wcurrent - (learning_rate) x gradient (GD)

한 batch 내 각 샘플에 대해 계산된 기울기들의 평균

샘플 1개당 기울기

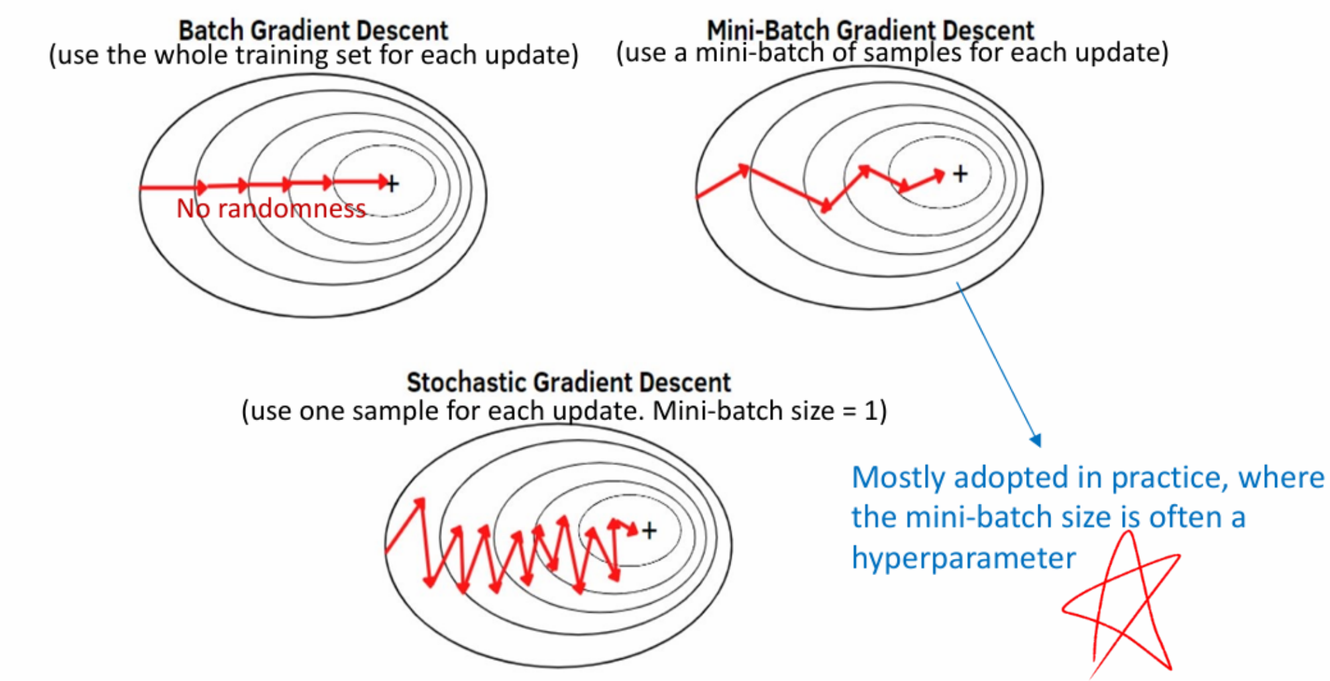
$$\frac{\partial L(w_t)}{\partial w_t} = \frac{1}{B}\sum_{k=1}^{B}\frac{\partial L_k(w_t)}{\partial w_t}$$

B: Batch size

$$w_{t+1} = w_t - \eta\frac{\partial L(w_t)}{\partial w_t}$$

Mini-Batch GD식
(Full-Batch와 SGD를 일반화)

경사하강법 비교



Batch size가 작다?

모델이 batch를 한 번 처리할 때 드는 연산량이 줄어듦

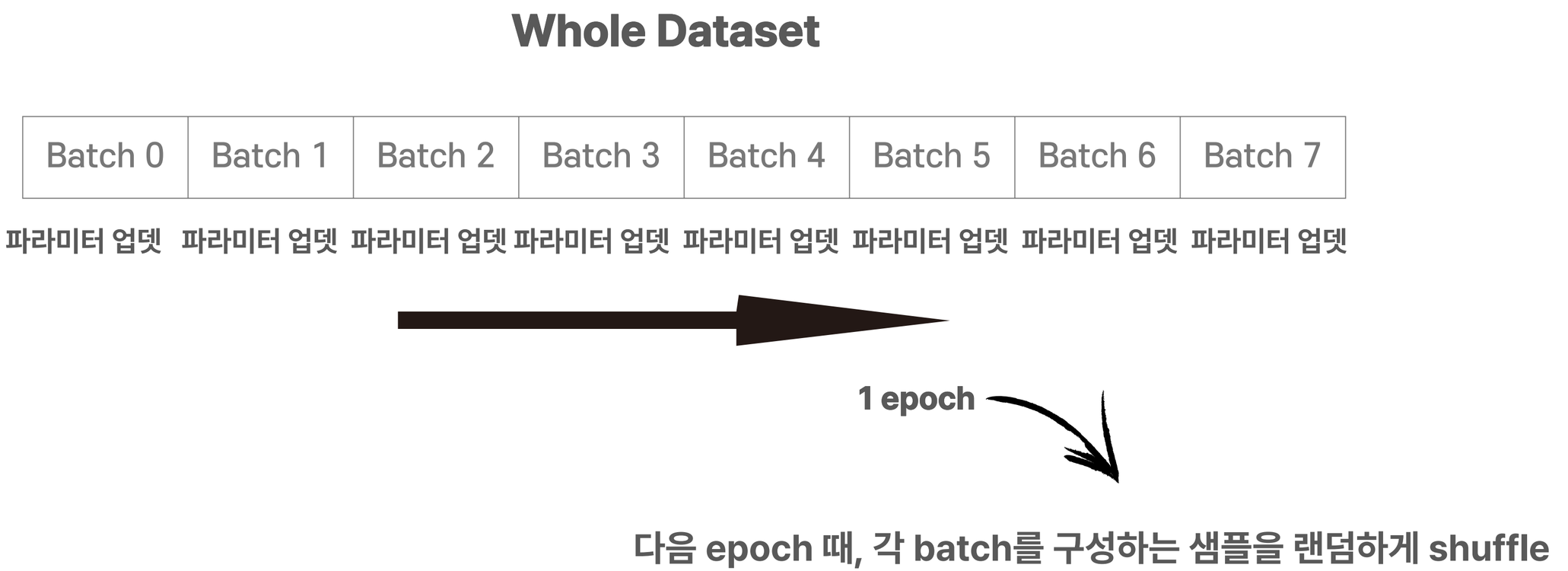
매번 적은 샘플로 파라미터를 업데이트하기 때문에 학습이 불안정
(GD식에 대입되는 gradient 방향이 매 시도마다 들쭉날쭉)

Batch size가 크다?

학습이 안정적

모델이 batch를 한 번 처리할 때 드는 연산량이 늘어남

배치를 활용한 학습 과정


Whole Dataset
Batch 0
Batch 1
Batch 2
Batch 3
Batch 4
Batch 5
Batch 6
Batch 7
파라미터 업뎃
파라미터 업뎃
파라미터 업뎃
파라미터 업뎃
파라미터 업뎃
파라미터 업뎃
파라미터 업뎃
파라미터 업뎃
1 epoch
다음 epoch 때, 각 batch를 구성하는 샘플을 랜덤하게 shuffle

Multi-Layer Perceptron (MLP): Backpropagation

파라미터를 업데이트하기 위해서는 gradient (d(Loss) / d(parameter))가 필요하다는 것은 알겠는데,

gradient는 어떻게 구하는 것일까??

→ Backpropagation

미적분학: Chain rule 기법을 사용

$$\frac{dy}{dx} = \frac{dy}{du} \times \frac{du}{dx}$$

<합성함수 미분>

y = f(u)
u = g(x)

y가 x를 파라미터로 가지지만 식이 분리되어 있어 y를 x에 대해 미분하지 못할 때 사용

Multi-Layer Perceptron (MLP): Backpropagation

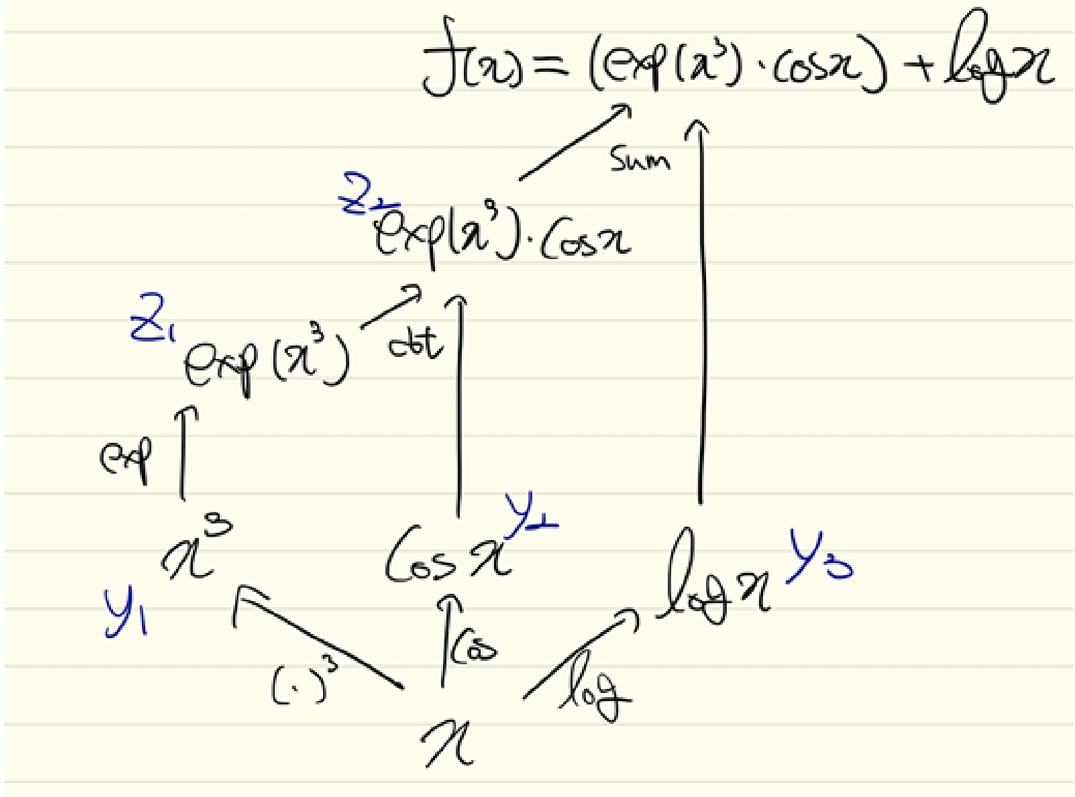
Computation Graph?

$$f(x) = ((\exp(x^3)) \cdot \cos x) + \log x$$

복잡한 합성함수가 계산되는 forward pass (input → output) 경로를 graph의 형태로 표현한 자료구조

Loss 역시 합성함수로 input → output으로 가는 forward pass가 존재

Pytorch(파이썬 딥러닝 프레임워크)는 Loss가 계산될 때 forward pass에 대해 computation graph를 동적 생성



Multi-Layer Perceptron (MLP): Backpropagation

Chain Rule 연습

f가 loss, x가 모델 파라미터라고 가정해보자

x를 업데이트하기 위해 필요한 gradient는 df / dx

Model's Forward Pass

x

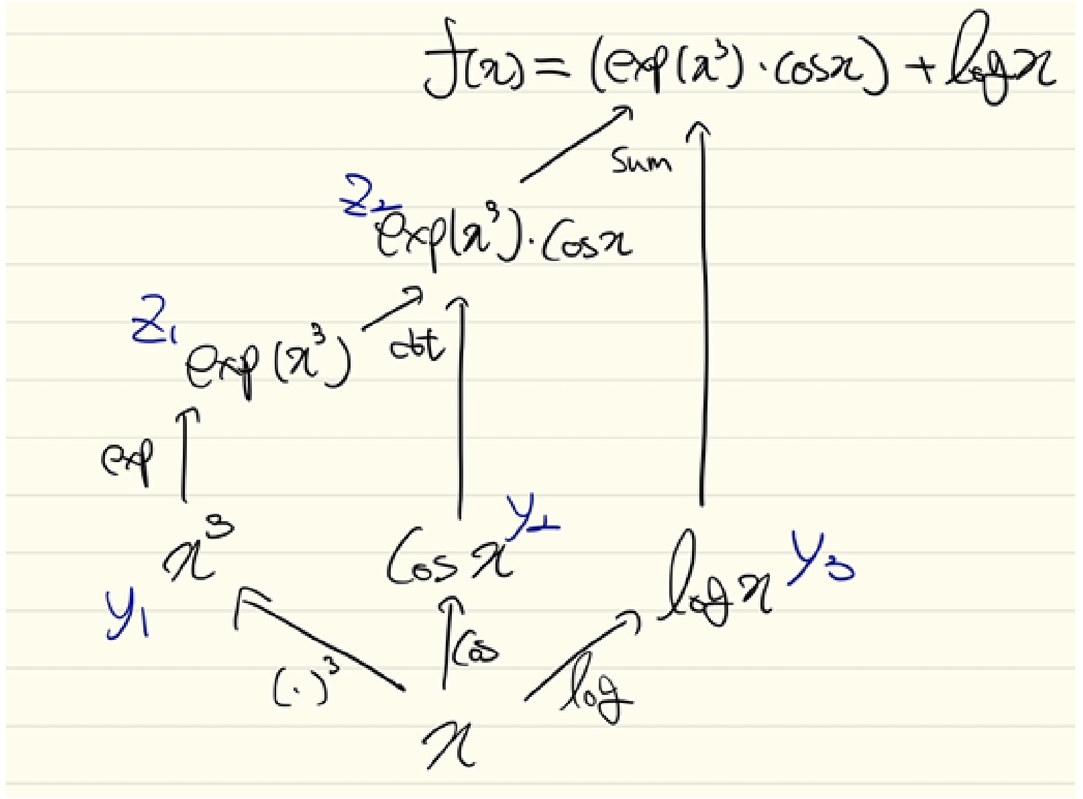
y1 = x^3 y2 = cosx y3 = logx

z1 = exp(y1)

z2 = z1 * y2

f = z2 + y3

$$f(x) = ((\exp(x^3)) \cdot \cos x) + \log x$$



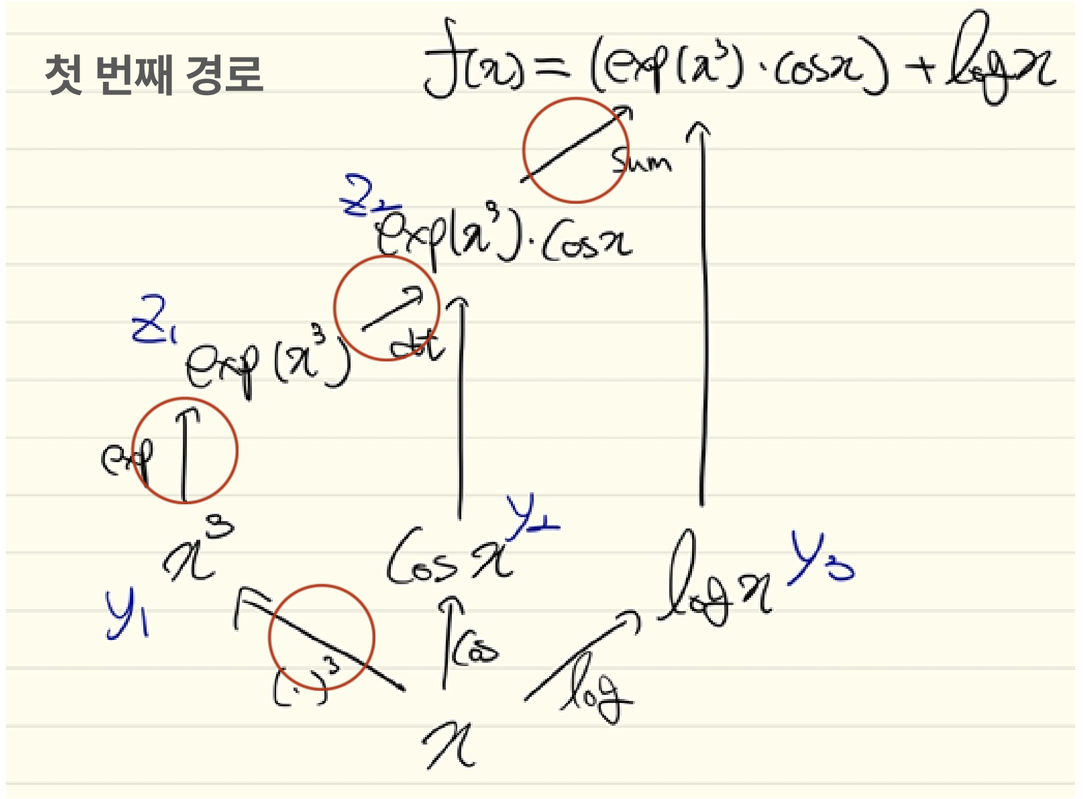
Multi-Layer Perceptron (MLP): Backpropagation

Chain Rule 연습

df / dx를 구해야 한다면, computation graph를 보았을 때,

f에서 x로 가는 모든 경로를 check

$$f(x) = ((\exp(x^3)) \cdot \cos x) + \log x$$



따라서,

df / dx (첫 번째 경로) = df / dz2 * dz2 / dz1 * dz1/dy1 * dy1/dx

뒤에서 앞으로: f → z2 → z1 → y1 → x

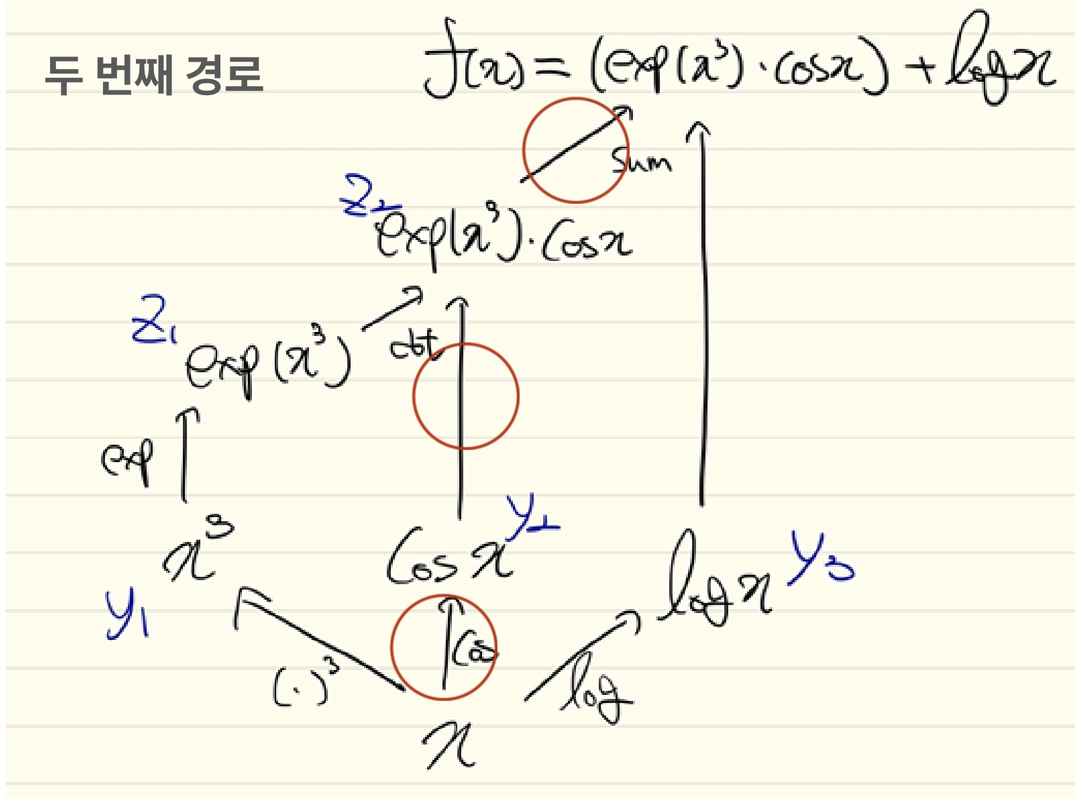
Multi-Layer Perceptron (MLP): Backpropagation

Chain Rule 연습

df / dx를 구해야 한다면, computation graph를 보았을 때,

f에서 x로 가는 모든 경로를 check

$$f(x) = ((\exp(x^3)) \cdot \cos x) + \log x$$



따라서,

df / dx (두 번째 경로) = df / dz2 * dz2 / dy2 * dy2/dx

뒤에서 앞으로: f → z2 → y2 → x

Multi-Layer Perceptron (MLP): Backpropagation

Chain Rule 연습

df / dx를 구해야 한다면, computation graph를 보았을 때,

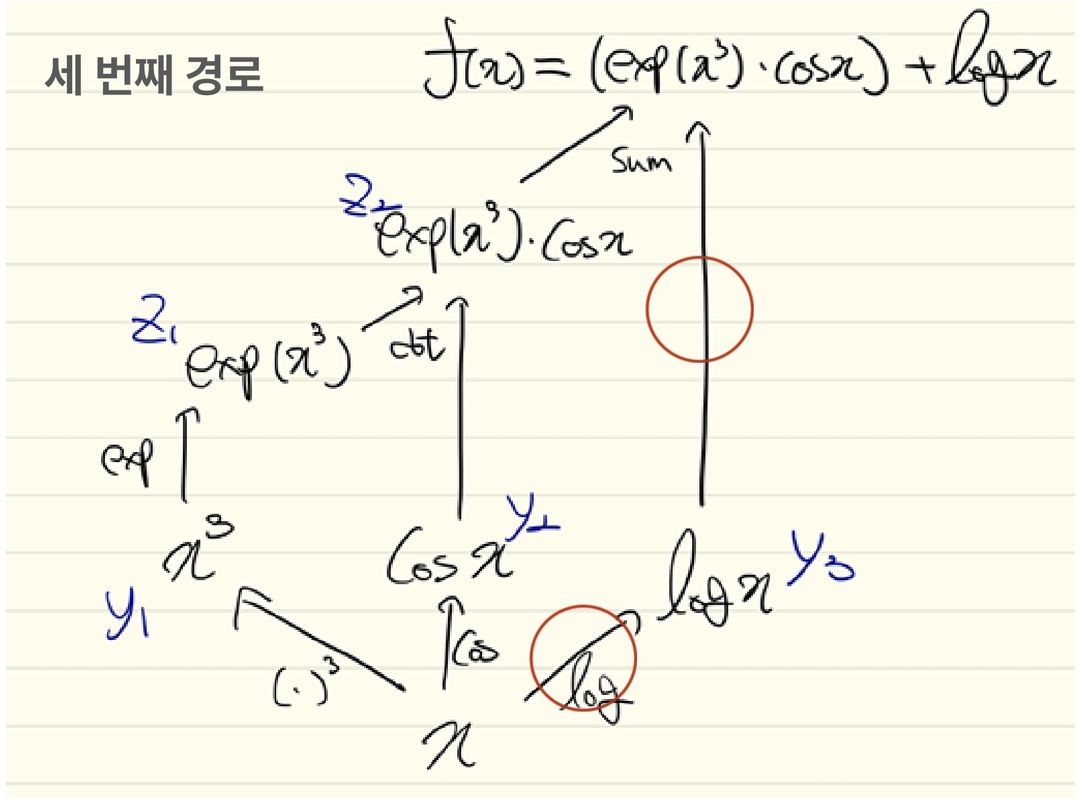
f에서 x로 가는 모든 경로를 check

따라서,

df / dx (세 번째 경로) = df / dy3 * dy3 / dx

뒤에서 앞으로: f → y3 → x

$$f(x) = ((\exp(x^3)) \cdot \cos x) + \log x$$



Multi-Layer Perceptron (MLP): Backpropagation

Chain Rule 연습

df / dx를 구해야 한다면, computation graph를 보았을 때,

f에서 x로 가는 모든 경로를 check

→ 모든 경로의 df / dx를 다 더해주면 최종 df / dx 값 완성

따라서,

df / dx = df / dx (첫 번째 경로) + df / dx (두 번째 경로) + df / dx (세 번째 경로)

= df / dz2 * dz2 / dz1 * dz1/dy1 * dy1/dx + df / dz2 * dz2 / dy2 * dy2/dx + df / dy3 * dy3 / dx

GD: X_next = X - (learning_rate) x df/dx

x

y1 = x^3 y2 = cosx y3 = logx

z1 = exp(y1)

z2 = z1 * y2

f = z2 + y3

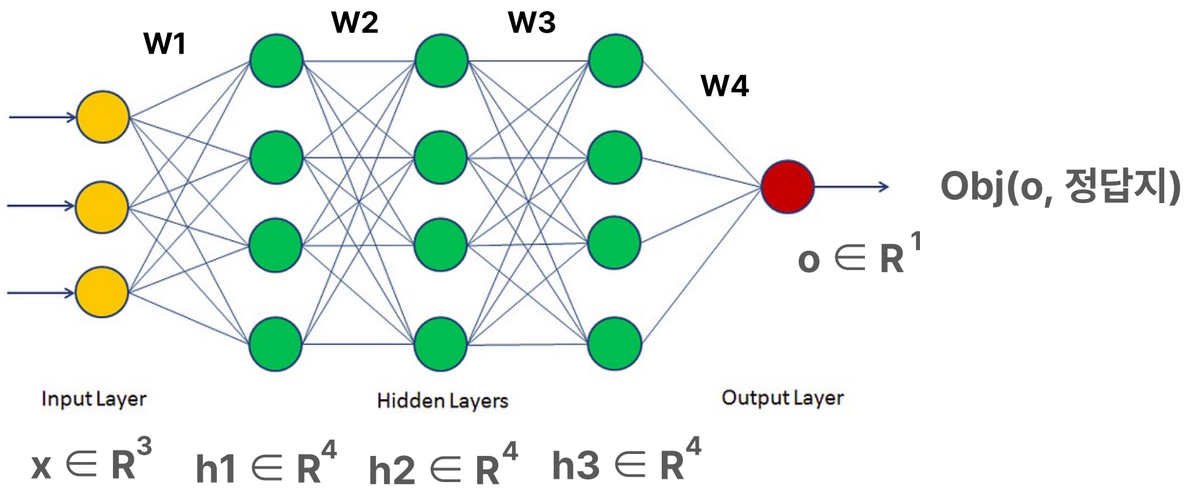
미분해서 적절한 값 구하기

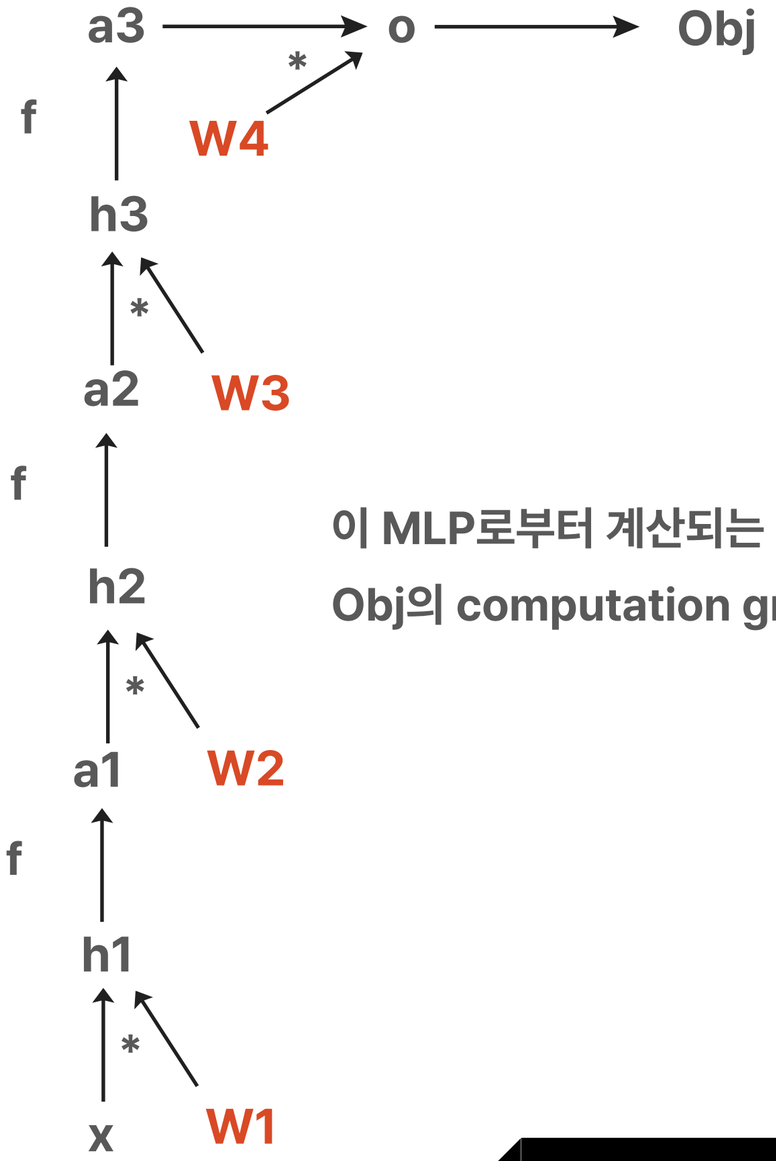
예를 들어, df / dz2 = 1

Multi-Layer Perceptron (MLP): Backpropagation

f: activation function

Obj: objective function (ex. loss)





이 MLP로부터 계산되는

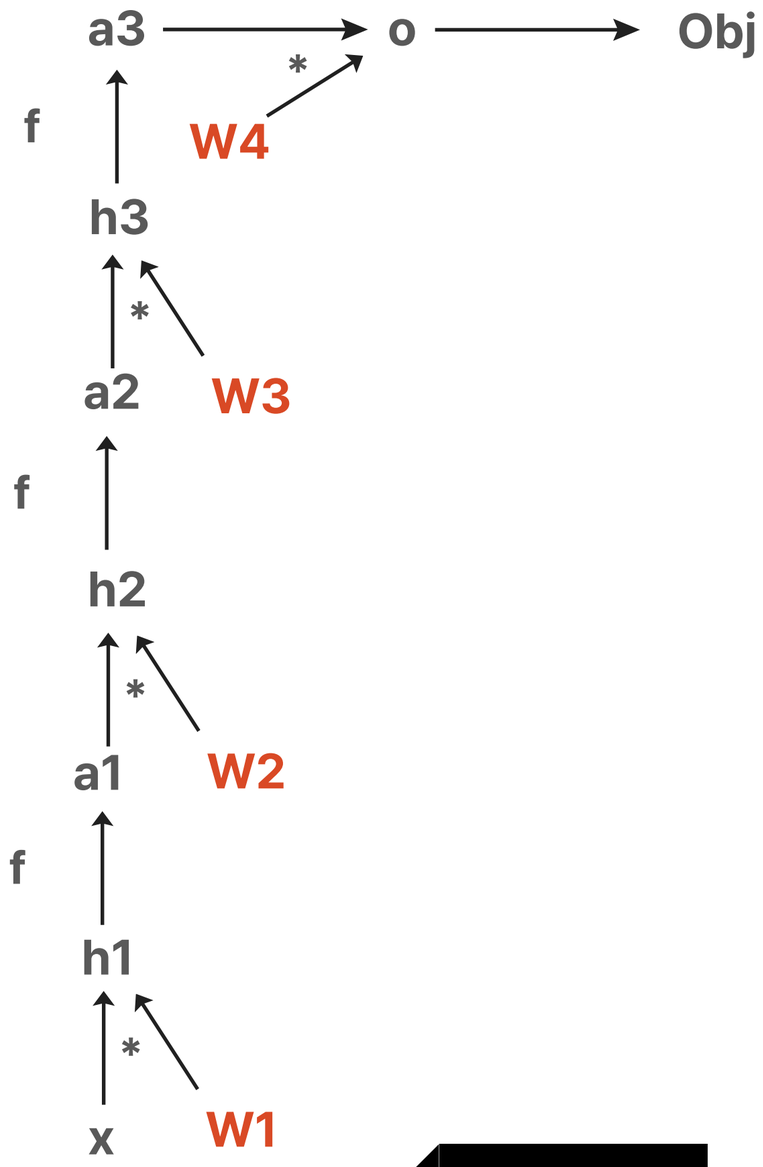
Obj의 computation graph

모든 파라미터(W1, W2, W3, W4)를 업데이트 하기 위해

d(obj)/dW1 d(obj)/dW2 d(obj)/dW3 d(obj)/dW4

를 각각 구해야함!

Multi-Layer Perceptron (MLP): Backpropagation



d(obj)/do 를 먼저 계산 (obj function은 o에 대해 표현된 식이기 때문에 미분하면 간단히 계산됨)

→ 해당 gradient는 **Activation gradient** 역할을 하게 됨

(GD식에 쓰이는 parameter gradient가 아닌 연산 절약을 위해 뒤에서 앞으로 전달되는 gradient)

Multi-Layer Perceptron (MLP): Backpropagation

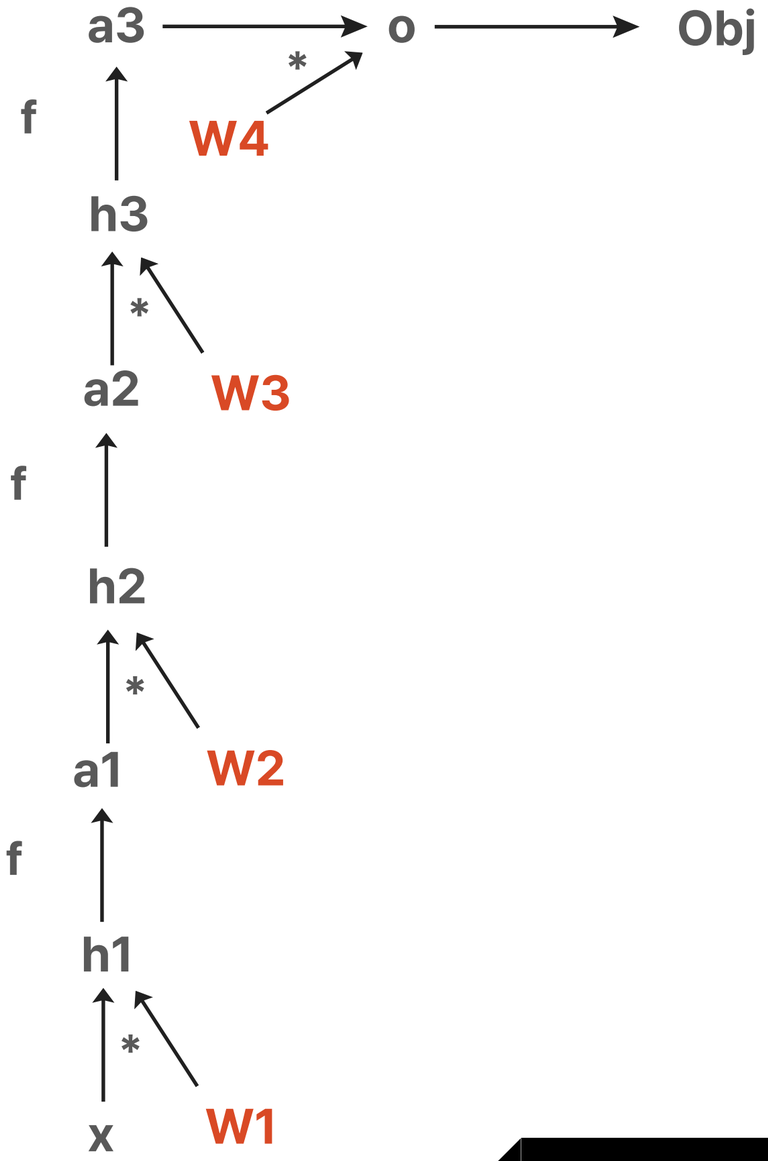
d(obj)/dW4 = d(obj)/do * do/dW4

o = W4 * a3

뒤에서 앞으로 전달된 activation gradient

= a3 (Forward pass에서 계산된 activation 값이 backpropagation 연산에 사용됨)

즉, forward pass에서 계산된 activation 값이 다음 layer의 input으로 들어간 후
바로 폐기되는게 아닌 memory에 저장되어야 함!!!



Multi-Layer Perceptron (MLP): Backpropagation

d(obj)/dW3를 계산해야하는데, 계산 전에

d(obj)/da3 를 먼저 계산 = d(obj)/do * do/da3

→ 해당 gradient는 **Activation gradient** 역할을 하게 됨

(GD식에 쓰이는 parameter gradient가 아닌 연산 절약을 위해 뒤에서 앞으로 전달되는 gradient)

Multi-Layer Perceptron (MLP): Backpropagation

d(obj)/dW3 = d(obj)/da3 * da3/dh3 * dh3/dW3

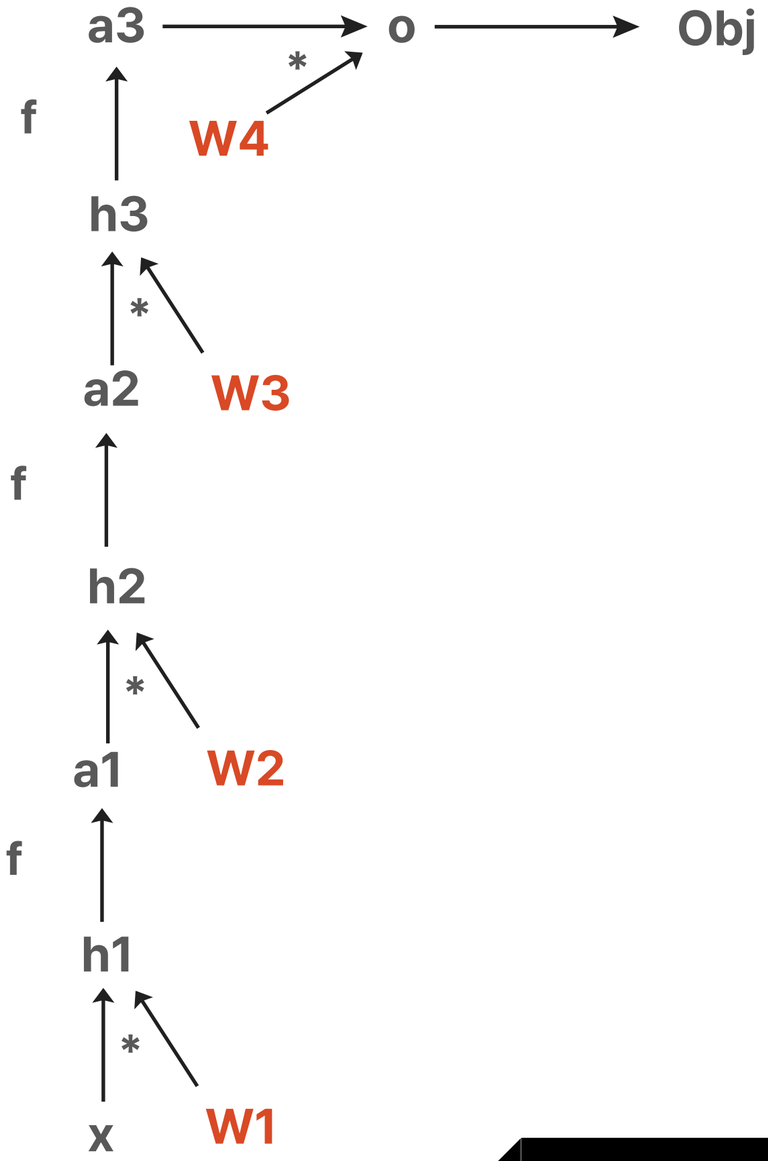
뒤에서 앞으로 전달된 activation gradient

f'(h3)

h3 = W3 * a2

= a2 (Forward pass에서 계산된 activation 값이 backpropagation 연산에 사용됨)

즉, forward pass에서 계산된 activation 값이 다음 layer의 input으로 들어간 후
바로 폐기되는게 아닌 backpropagation에서 사용 전까지 memory에 저장되어야 함!!!



Multi-Layer Perceptron (MLP): Backpropagation

정리 1

역전파(Backpropagation) 시, 계산되는 gradient는 두 종류

파라미터 업데이트 용이 아닌 연산 절약을 위해 뒤에서 앞으로 전달되는 gradient
(저는 activation gradient라고 부릅니다.)

파라미터 업데이트 용 gradient: GD 식에 넣는 gradient
(저는 parameter gradient라고 부릅니다.)

Multi-Layer Perceptron (MLP): Backpropagation

정리 2

Forward pass할 때 계산되는 activation 값들은 다음 layer의 input으로 들어가고 바로 폐기되는 것이 아닌 Backpropagation 연산에 재활용해야하기 때문에 일시적으로 memory에 저장됨

따라서 모델 돌리면서 GPU 메모리가 터졌을 때, 모델의 크기뿐만 아니라 activation memory도 고려해야함 (세부적인 디테일은 마지막 주차 분산학습 파트에서 다루겠습니다.)

Advanced Gradient Descent

사실, 대부분의 학습에서 GD만 사용하는 경우는 거의 없음

Advanced technique이 존재!

(S)GD with momentum → RMSProp → Adam → AdamW

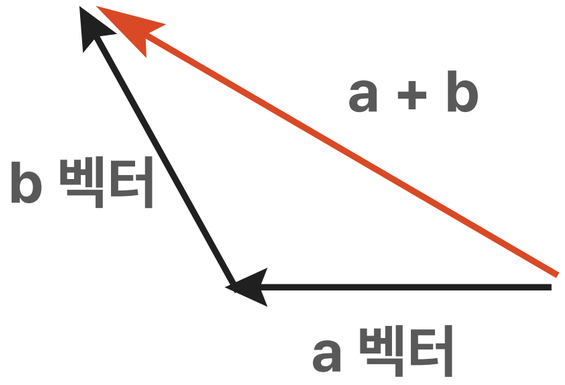
AdamW만 사용하면 되는데 이를 이해하기 위해서 나머지 기법들도 공부 필요

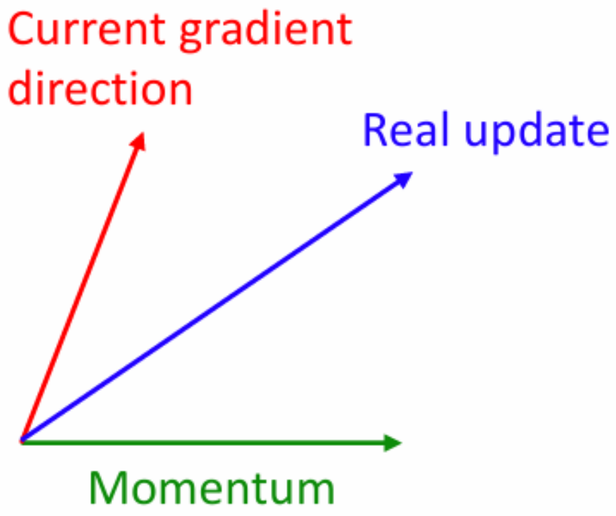
(S)GD with Momentum

Batch size가 크지 않으면, 매번 연산되는 gradient의 방향이 휙휙 바뀔 수 있어 학습이 불안정

Gradient 방향이 너무 휙휙 바뀌지 않게 이전 gradient 방향을 일부 반영하여 새 gradient를 생성 가능

→ Sum of vector를 사용

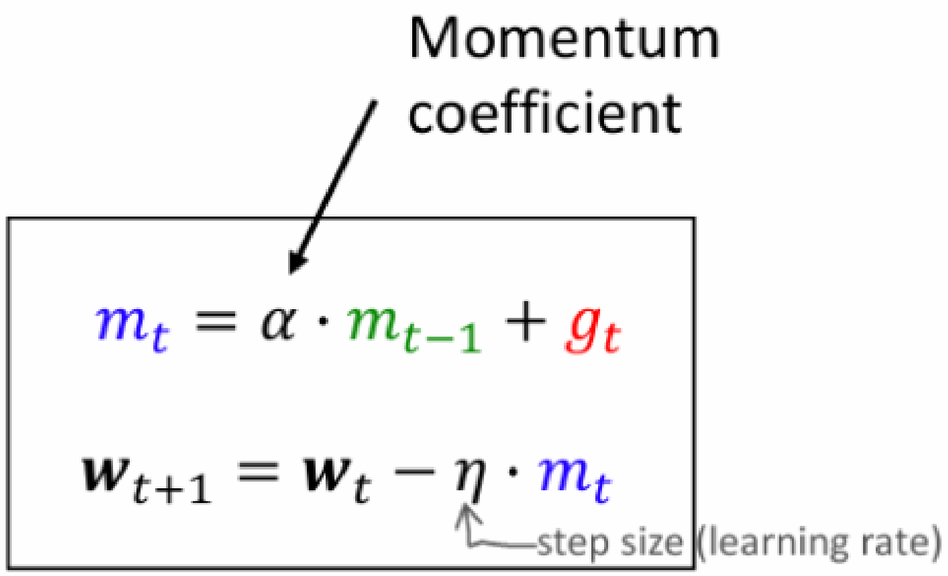




(S)GD with Momentum

mt는 연산되는 기울기를 차곡 차곡 쌓은 누적 기울기라고 생각

→ 이 값을 GD 식에 넣어 계산



m0 = 0 (누적된 기울기가 없는 상태)

m1 = α x m0 + g1

m2 = α x m1 + g2 = α^2 x m0 + α x g1 + g2

...

mt = α^t x m0 + α^{t-1} x g1 + ... + gt

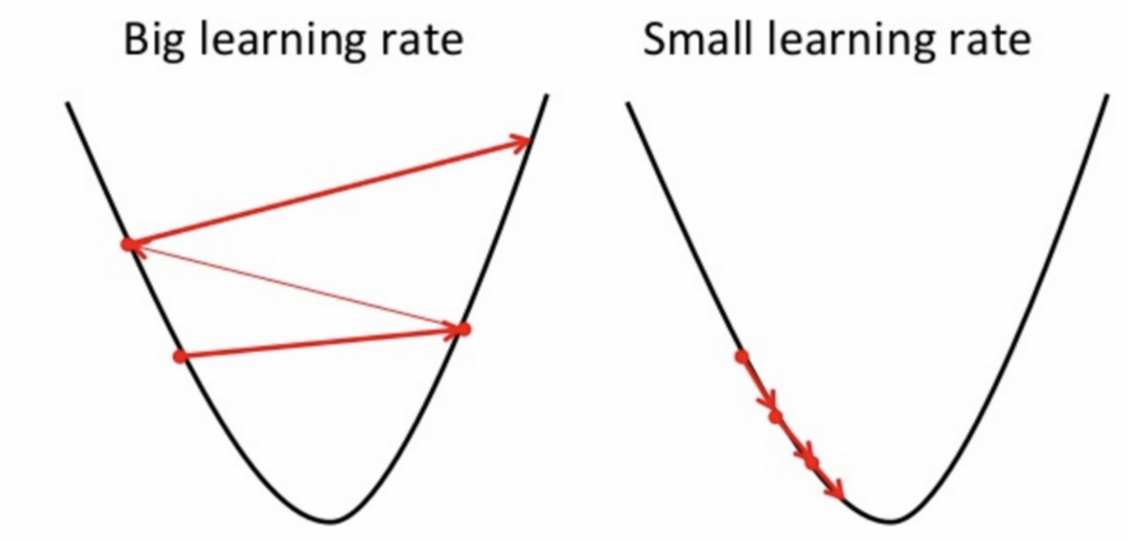
α: 0에서 1 사이 값

과거의 기울기일수록 sum of vector에서 덜 반영됨

RMSProp

최근 gradient의 크기가 너무 커서 큰 보폭으로 업데이트가 되었다면, learning rate를 동적으로 줄임

최근 gradient의 크기가 너무 작아 작은 보폭으로 업데이트가 되었다면, learning rate를 동적으로 늘림



RMSProp

Vt는 기울기 제곱을 누적한 값 → 이 값을 고려해 learning rate 조절

기울기 제곱을 누적하는 이유는 부호(+/-) 고려 없이 크기 정보만을 담기 위해서

$$v_t = \beta v_{t-1} + (1-\beta)(g_t)^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t} + \epsilon} \cdot g_t$$

Vt가 클수록 전체 learning rate 감소

(루트를 씌운 이유는 부호 없애느라 기울기를 제곱했기 때문)

$V0 = 0$

$V1 = \beta \times V0 + (1-\beta)(g1)^2$

$V2 = \beta \times V1 + (1-\beta)(g2)^2 = \beta^2 \times V0 + \beta(1-\beta)(g1)^2 + (1-\beta)(g2)^2$

...

$Vt = \beta^t \times V0 + \beta^{t-1}(1-\beta)(g1)^2 + \beta^{t-2}(1-\beta)(g2)^2 + \dots (1-\beta)(gt)^2$

과거의 기울기일수록 그 크기 정보가 덜 반영됨

β: 0에서 1 사이 값

Adam

Momentum 개념과 동적 learning rate 조절(Adaptive learning rate) 개념을 합침

$$m_t = \alpha \cdot m_{t-1} + (1-\alpha) \cdot g_t$$

$$v_t = \beta \cdot v_{t-1} + (1-\beta) \cdot (g_t)^2$$

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \cdot \hat{m}_t$$

mt와 vt를 바로 쓰지 않고 이렇게 조정한다음 쓰는 이유(참고)

$$\hat{m}_t = \frac{m_t}{1-\alpha^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta^t}$$

Bias-corrected first and second moments
(second moment = moving average of gradient square)

At time 1, for example, $m_1 = (1-\alpha)g_1$, which is off by the factor $(1-\alpha)$ from the true gradient, so we want to correct this bias.

AdamW

Adam + Weight decay(L2 규제 적용)

Adam 식 그대로 계산 이후, GD 식에 넣을 때

$$\theta_t = \theta_{t-1} - \eta \cdot \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \cdot \theta_{t-1} \right)$$

L2 Regularization (1주차 참고)

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{task}}(\theta) + \frac{\lambda}{2}\|\theta\|_2^2$$

Overfitting 방지

미분하면 λ* θ

모델 안정화: Batch Normalization

Layer n의 output이 Layer n+1의 input으로

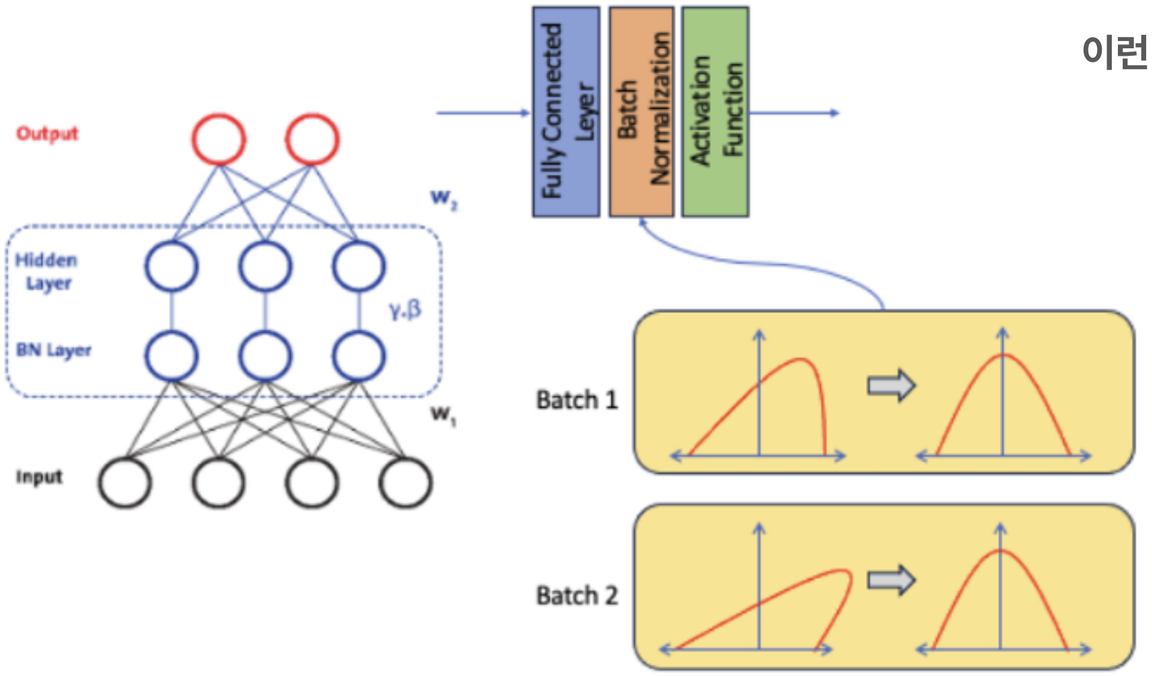
파라미터가 업데이트 또는 배치가 바뀜에 따라 layer들의 출력 및 입력 분포가 바뀜

이러면 다음 layer도 영향 받아 모델의 학습이 불안정해짐

Layer n의 출력분포가 바뀌면 바뀐 분포가 n+1 layer의 입력 분포가 되어 학습 난이도가 높아짐

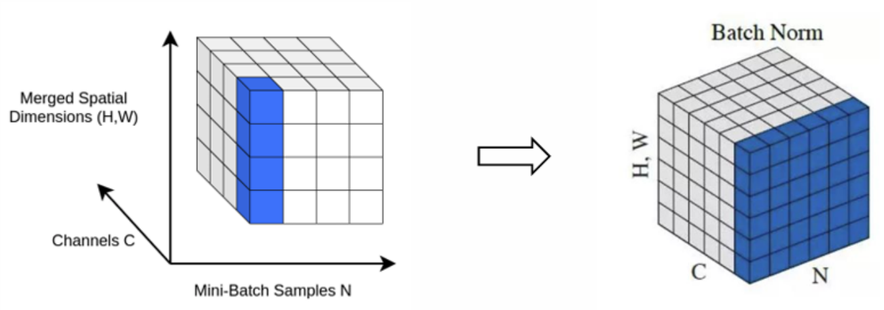
Layer n의 출력분포를 받아 일정한 분포를 내뱉는 layer를 하나 더 두면 n + 1 layer의 입력 분포는 안정화됨

모델 안정화: Batch Normalization



이런 식으로 layer 사이에 넣어 batch에 따라 분포의 차이가 없도록 안정화

모델 안정화: Batch Normalization



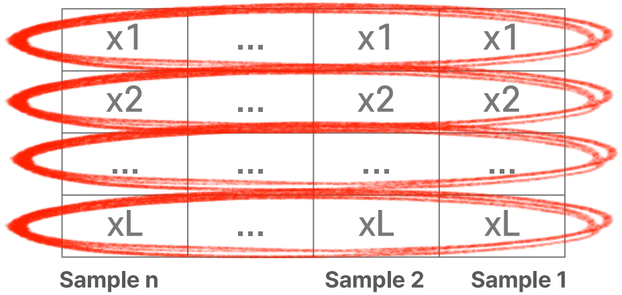
$$\hat{x} = \frac{x - \mu_B}{\sigma_B}$$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2$$

평균과 표준편차 계산

이 범위 내 값들로 평균과 표준편차 계산

One batch size = n



Batch Norm

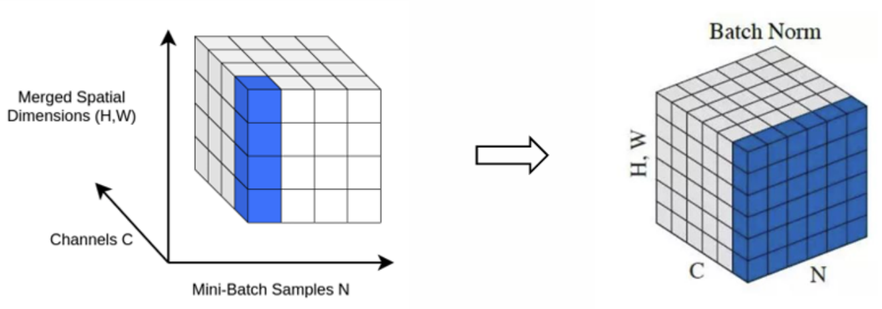
$$y = \gamma \odot \hat{x} + \beta$$

y: Batch Normalization의 최종 output

평균이 0, 표준편차가 1인 정규분포가 바람직한 정규분포가 아닐 수 있기 때문에

Y와 β라는 학습 파라미터를 둠

모델 안정화: Batch Normalization



$$\hat{x} = \frac{x - \mu_B}{\sigma_B}$$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2$$

평균과 표준편차 계산

Inference할 때는 배치가 아닌 한 샘플을 input으로 넣는 건데 BN layer는 어떻게 연산되나요?

Training동안 평균과 표준편차를 축적해서 저장해놓고, 해당 값들을 그대로 식에 대입

$$\text{running_mean} \leftarrow (1 - m) \cdot \text{running_mean} + m \cdot \mu_B$$

$$\text{running_var} \leftarrow (1 - m) \cdot \text{running_var} + m \cdot \sigma^2_B$$

이 범위 내 값들로 평균과 표준편차 계산

One batch size = n

x1	...	x1	x1
x2	...	x2	x2
...	...	...	...
xL	...	xL	xL
Sample n		Sample 2	Sample 1

Batch Norm

모델 안정화: Residual Connection

잔차연결 (Residual Connection)

$$x_{L+1} = x_L + f(x_L)$$

최종 output　f의 input　output

input을 output에 더해주는 연산

<등장 배경>

Backpropagation에서 chain rule에 따라 특정 기울기가 다른 기울기들을 연속해서 곱해서 구하는데, 만약 계산에 참여하는 기울기의 값이 작으면 결과 기울기 값이 굉장히 작을 수 있음

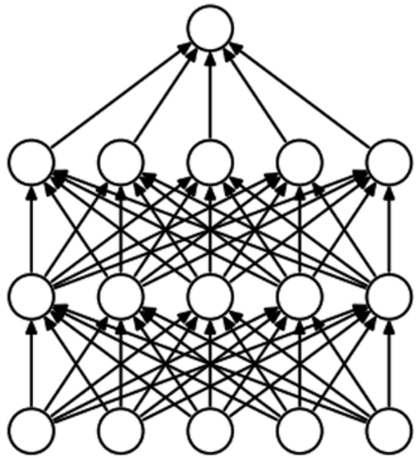
→ 해당 파라미터의 업데이트가 더뎌짐 (Vanishing Gradient)

scNOVA에서 사용하고 있는 ResNet은 Residual connection 기법을 도입한 모델 중 하나

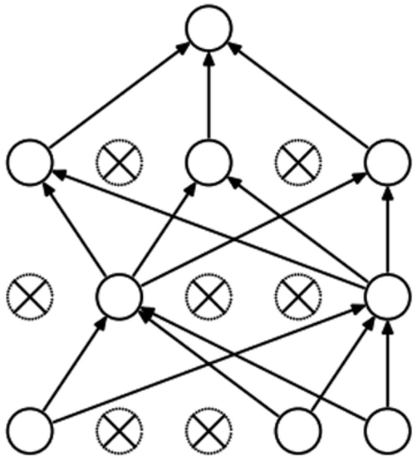
오버피팅 방지: Dropout

은닉층 값의 일부를 랜덤하게 0으로 만듦

→ 너무 훈련데이터에만 과적합하지 않게 방지



(a) Standard Neural Net



(b) After applying dropout.

Training

stochastic

$$
\begin{aligned}
r_j^{(l)} &\sim \text{Bernoulli}(p), \\
\tilde{\mathbf{y}}^{(l)} &= \mathbf{r}^{(l)} * \mathbf{y}^{(l)}, \\
z_i^{(l+1)} &= \mathbf{w}_i^{(l+1)} \tilde{\mathbf{y}}^{l} + b_i^{(l+1)}, \\
y_i^{(l+1)} &= f(z_i^{(l+1)}).
\end{aligned}
$$

베르누이 확률 분포에서 0 or 1을 p와 1-p의 확률로 샘플링

해당 r값을 은닉층 값에 곱해줌 (그러면 y는 0 또는 y 값을 가지게 됨)

→ 값을 확률적으로 0으로 직접 만듦

Inference

deterministic

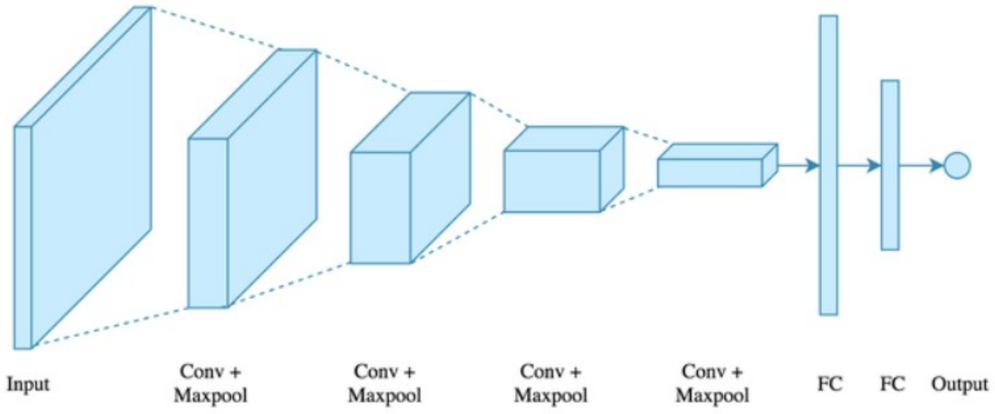
$$
\begin{aligned}
r_j^{(l)} &\sim \text{Bernoulli}(p), \\
\tilde{\mathbf{y}}^{(l)} &= p * \mathbf{y}^{(l)}, \\
z_i^{(l+1)} &= \mathbf{w}_i^{(l+1)} \tilde{\mathbf{y}}^{l} + b_i^{(l+1)}, \\
y_i^{(l+1)} &= f(z_i^{(l+1)}).
\end{aligned}
$$

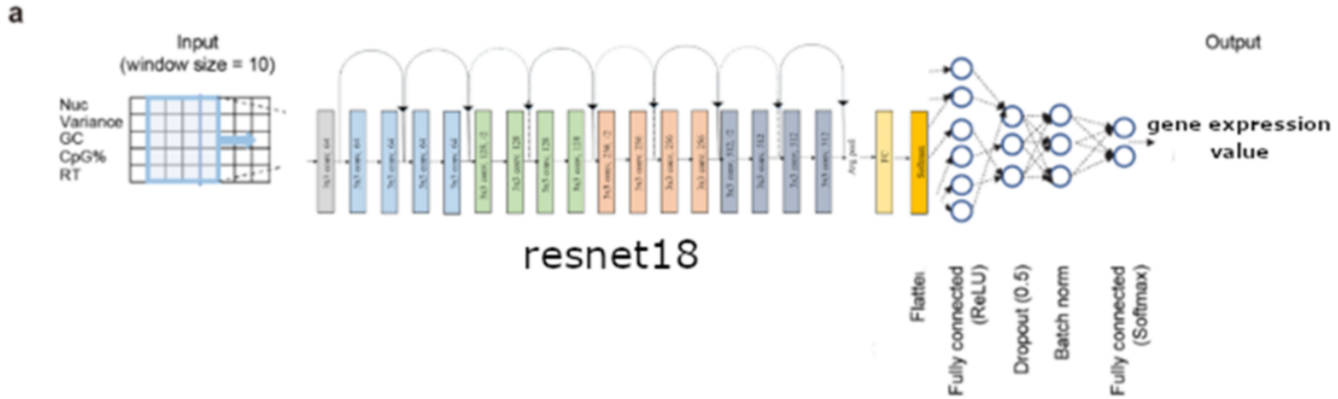
p == 베르누이 분포의 평균

inference할 때는 y에 p를 곱해서 training과 scale을 맞춤

(값을 없애는게 아니라 확률을 곱해서 출력 scale을 맞춰줌)

CNN


Input
Conv + Maxpool
Conv + Maxpool
Conv + Maxpool
Conv + Maxpool
FC
FC
Output


a
Input
(window size = 10)
Nuc
Variance
GC
CpG%
RT
resnet18
Output
gene expression value
Flatten
Fully connected (ReLU)
Dropout (0.5)
Batch norm
Fully connected (Softmax)

scNOVA

CNN: 등장 배경

MLP 단점

1. 한 데이터 샘플의 크기가 커질수록 파라미터 수가 급격히 커짐

   ex. 224 x 224 x 3 이미지를 MLP 첫 layer에 넣어서 10000차원으로 바꾼다고 치면

   첫 번째 layer 만 해서 224 x 224 x 3 x 10000 = 1505280000개의 파라미터가 필요

2. 한 샘플이 MLP에 들어가려면 무조건 벡터 형태로 바꿔야 함

   → 특히 이미지 같은 경우 2d matrix 모양을 벡터로 바꿔버리면 중요한 정보를 놓칠 수 있음

CNN: Filter

CNN은 Filter를 학습 파라미터로 둠...

그렇다면 filter란?

컴퓨터가 이미지를 이해하는 방법을 탐구하는 컴퓨터 비전(CV) 분야에서 사용되는 개념

컴퓨터가 filter를 이미지 위에서 스캔하듯이 이동시키면서, 해당 filter를 통해 이미지의 feature를 추출하거나 이미지의 특정 부분을 강조

CNN: Filter Calculation 1 (Cross-Correlation)

$$G[i,j] = \sum_{u=-k}^{k} \sum_{v=-k}^{k} H[u,v]F[i+u, j+v]$$

output filter image

Cross-Correlation 연산

2	4	0	1
1	1	2	3
5	4	4	2
3	3	2	1

image

2	3
4	5

filter

25		

2 x 2 + 4 x 3 + 1 x 4 + 1 x 5

CNN: Filter Calculation 1 (Cross-Correlation)

$$G[i,j] = \sum_{u=-k}^{k} \sum_{v=-k}^{k} H[u,v]F[i+u, j+v]$$

output　filter　image

Cross-Correlation 연산

2	4	0	1
1	1	2	3
5	4	4	2
3	3	2	1

image

2	3
4	5

filter

25	22	

4 x 2 + 0 x 3 + 1 x 4 + 2 x 5

CNN: Filter Calculation 1 (Cross-Correlation)

$$G[i,j] = \sum_{u=-k}^{k} \sum_{v=-k}^{k} H[u,v]F[i+u, j+v]$$

output　　filter　　image

Cross-Correlation 연산

2	4	0	1
1	1	2	3
5	4	4	2
3	3	2	1

image

2	3
4	5

filter

25	22	26

0 x 2 + 1 x 3 + 2 x 4 + 3 x 5

CNN: Filter Calculation 1 (Cross-Correlation)

$$G[i,j] = \sum_{u=-k}^{k} \sum_{v=-k}^{k} H[u,v]F[i+u, j+v]$$

output　filter　image

Cross-Correlation 연산

2	4	0	1
1	1	2	3
5	4	4	2
3	3	2	1

image

2	3
4	5

filter

25	22	26
45		

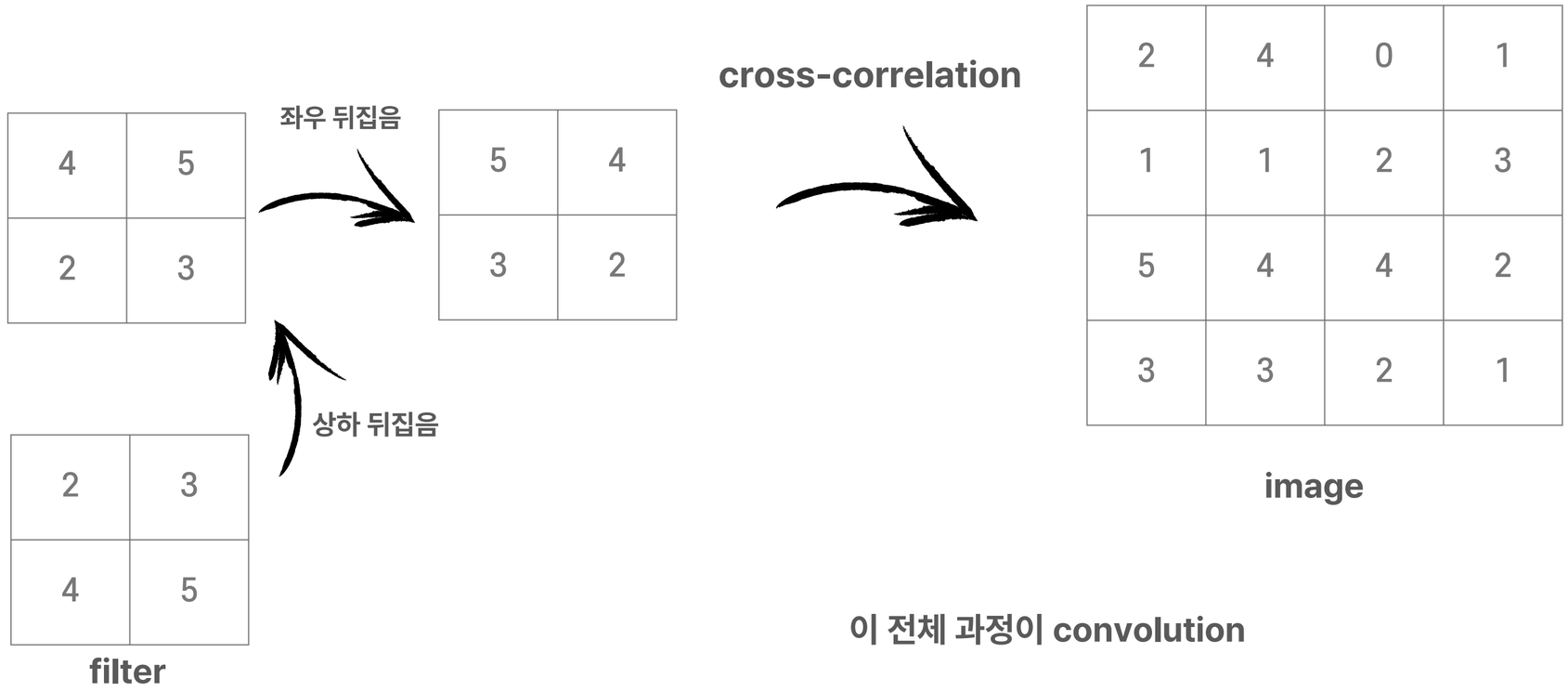
1 x 2 + 1 x 3 + 5 x 4 + 4 x5

CNN: Filter Calculation 2 (Convolution)

$$G[i,j] = \sum_{u=-k}^{k} \sum_{v=-k}^{k} H[u,v]F[i-u, j-v]$$

output filter image

Convolution 연산



이 전체 과정이 convolution

CNN: Filter 예시

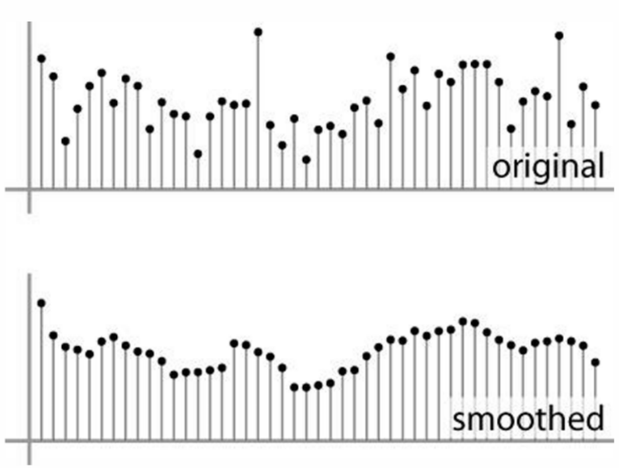
이 필터로 cross-correlation 연산 취하면, 해당 구간 내 이미지 픽셀들 사이에 가중 평균이 구해짐

그럼 필터 연산의 output은 픽셀들의 평균으로 이루어진 새로운 픽셀들로 구성됨

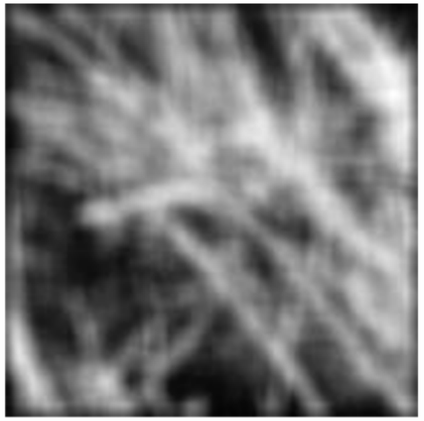
Gaussian Filter

$$\frac{1}{16}\begin{bmatrix}1 & 2 & 1\\ 2 & 4 & 2\\ 1 & 2 & 1\end{bmatrix}$$

$$H[u, v]$$



original



filtered

평균냄으로써 자기 혼자 intensity가 튀는 noise 픽셀들의 intensity를 줄일 수 있음

CNN: Filter

이런 식으로 filter는 이미지로부터 어떤 feature를 뽑아내거나, 이미지의 특정 부분을 강조하거나 없앨 수 있는 기능이 있음

x	y
z	h

filter

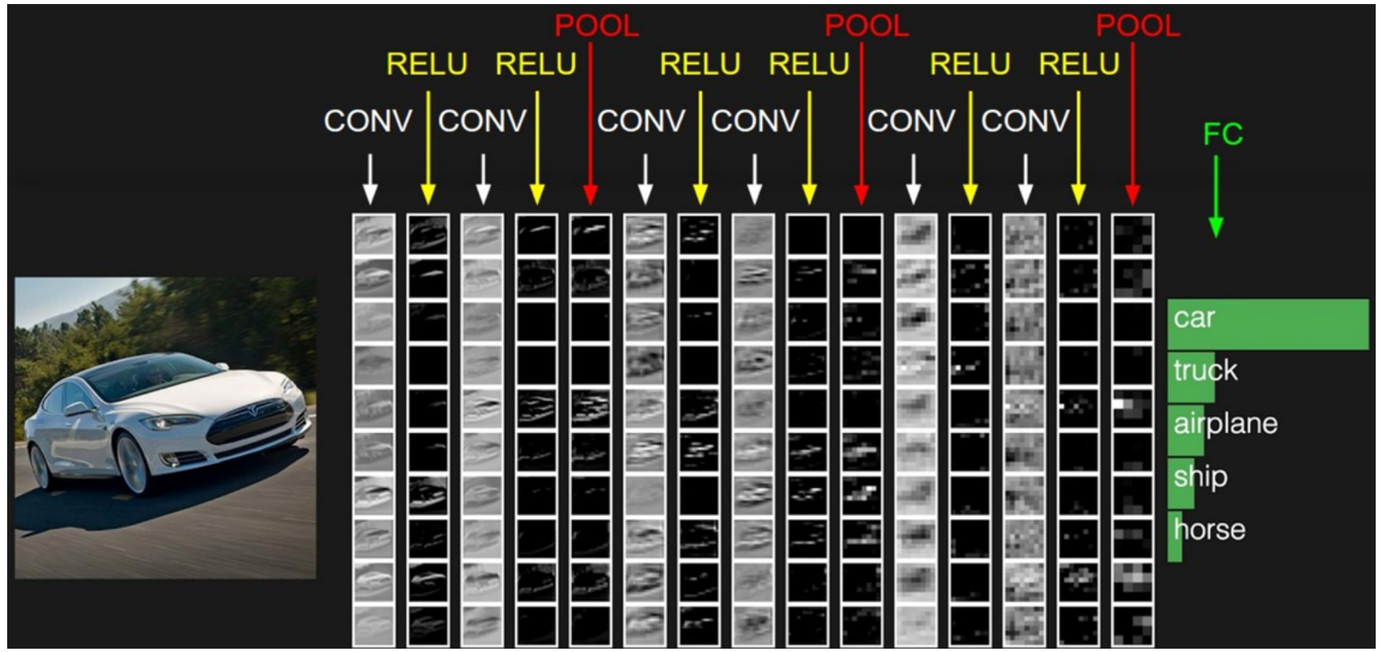
CNN은 이 filter가 학습 파라미터

값이 바뀌는 학습 파라미터이기 때문에, 상하좌우로 뒤집는게 의미가 없음

CNN에서 사용하는 filter 연산은 cross-correlation이지만 convolution이라고 그냥 부름

CNN

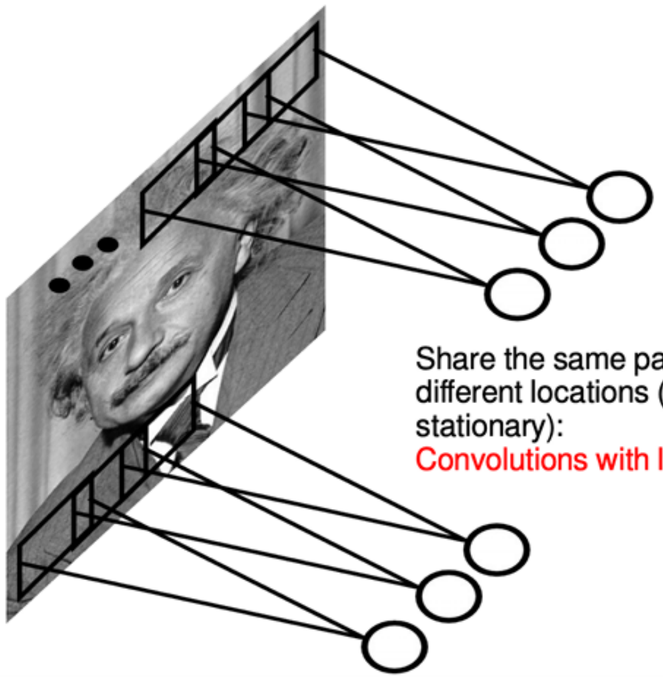
CNN은 크게 3가지 layer로 구성: Convolution layer, Pooling layer, Fully-connected layer



Conv+Pooling의 반복 → FC layer로 마무리

CNN: Convolution layer

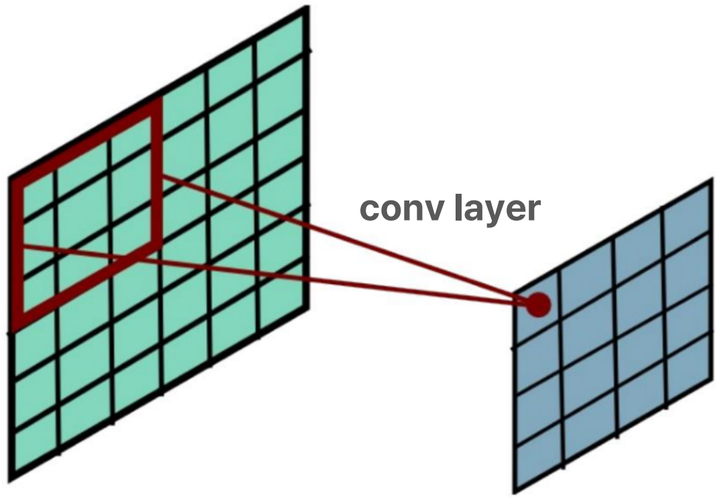
filter를 이미지 전체를 스캔하듯이 이동하면서 convolution 연산 수행 → 해당 이미지의 feature를 추출



Share the same parameters across different locations (assuming input is stationary):
Convolutions with learned kernels

파라미터 수를 아낄 수 있음

kernel(filter의 다른 말)

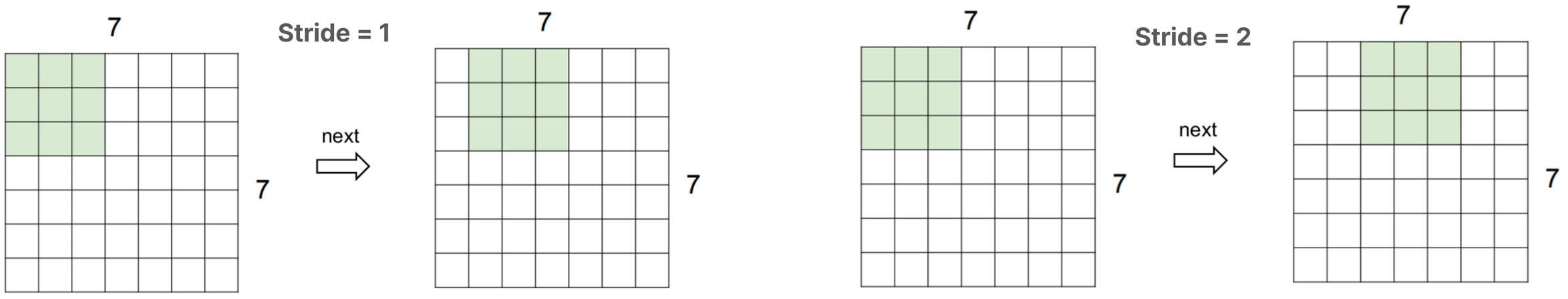


학습된 filter를 거치면서
뭔지는 모르겠지만, 우리 task에 필요한 feature를 추출

CNN: Convolution layer

filter를 이미지 전체를 스캔하듯이 이동하면서 convolution 연산 수행 → 해당 이미지의 feature를 추출

Stride란? Filter의 이동 보폭

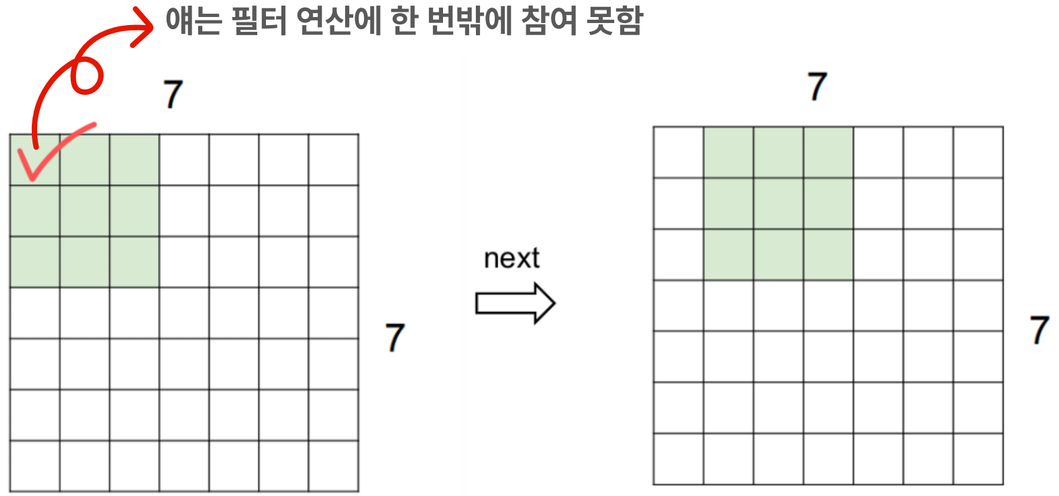


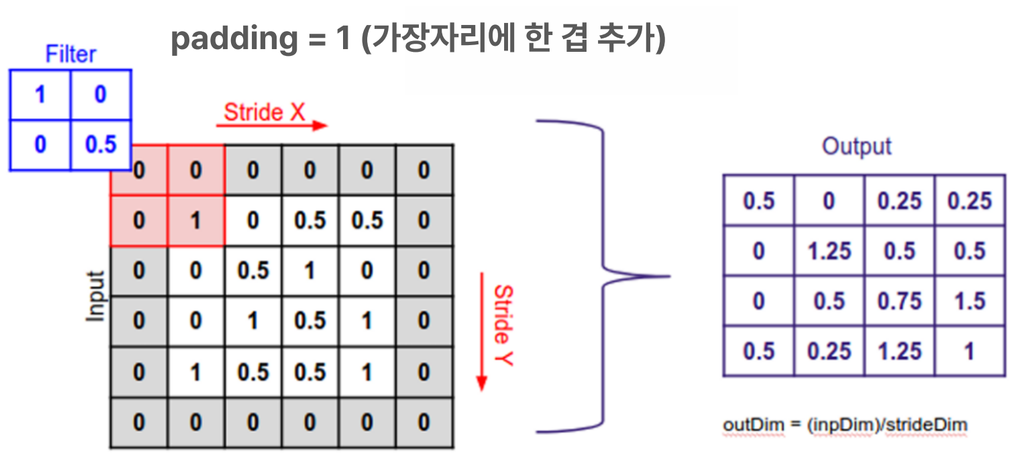
CNN: Convolution layer

filter를 이미지 전체를 스캔하듯이 이동하면서 convolution 연산 수행 → 해당 이미지의 feature를 추출

Padding이란?

가장자리에 존재하는 픽셀의 정보가 덜 반영되는 문제를 해결하기 위해 가장자리에 더미 픽셀을 추가

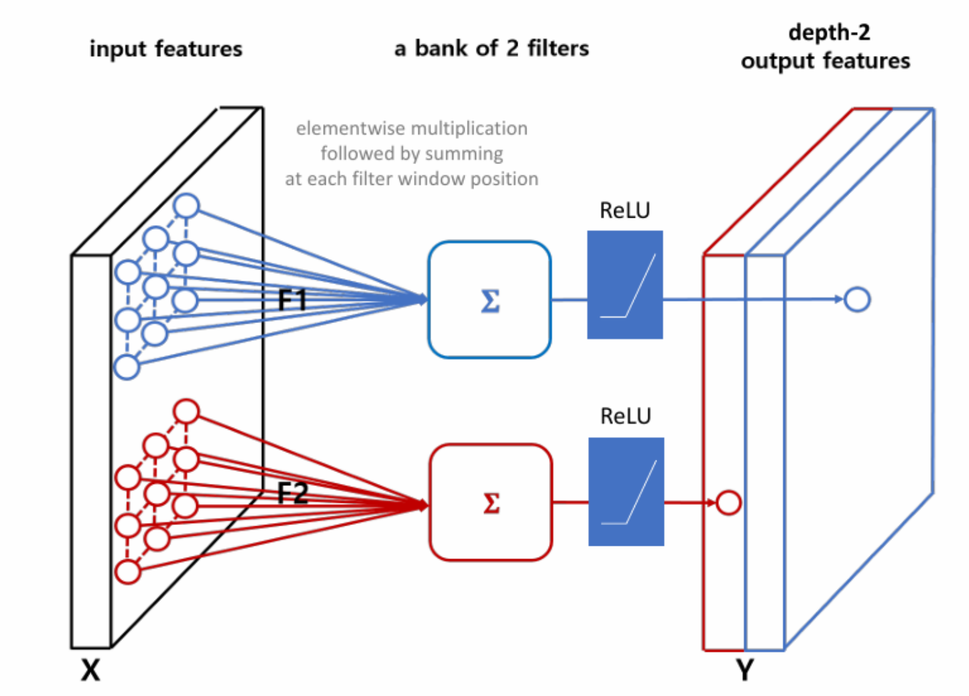




zero padding: 가장자리에 0들을 추가

CNN: Multiple Filter

여러 개의 filter가 동일 조건(padding, stride 등)으로 convolution 수행 → feature map이 filter 개수만큼 생성



filter 2개 convolution 수행 → feature map이 2개 생성

input이 3차원(width x height x depth)일 때,
filter의 depth와 input의 depth는 같아야 함!!

CNN: Convolution Layer 차원 설계

거듭 강조하지만, 딥러닝에서 차원 계산은 매우 중요!

Input tensor의 차원: W x H x D ex) 컬러(RGB) 이미지: 224 x 224 x 3

Filter의 차원: F x F x D x K (K는 filter 개수)

stride = s

padding = p

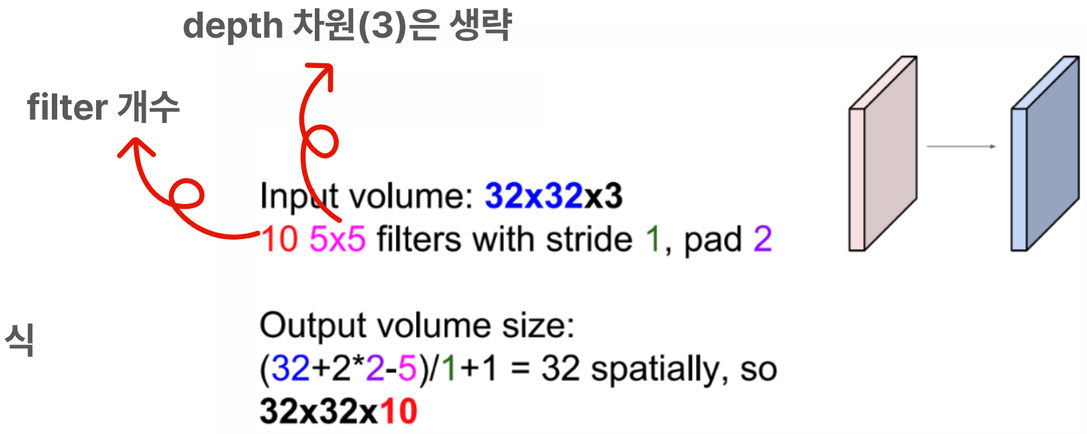
Output의 차원: W2 x H2 x D2 라고 하면,

W2 = ((W - F + 2p)/s) + 1

H2 = ((H - F + 2p)/s) + 1

D2 = K

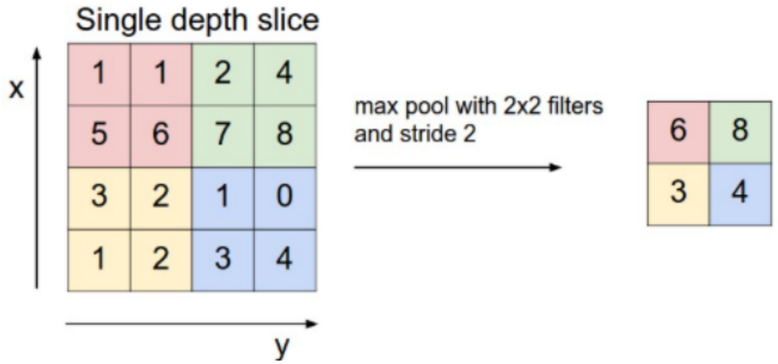
알아두면 좋은 식



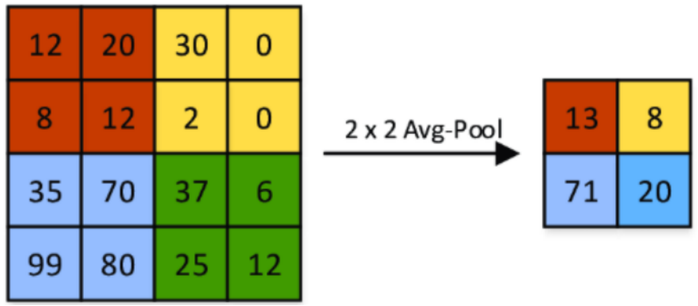
CNN: Pooling Layer

학습하는 layer는 아님

Max Pooling (많이 사용)



Average Pooling



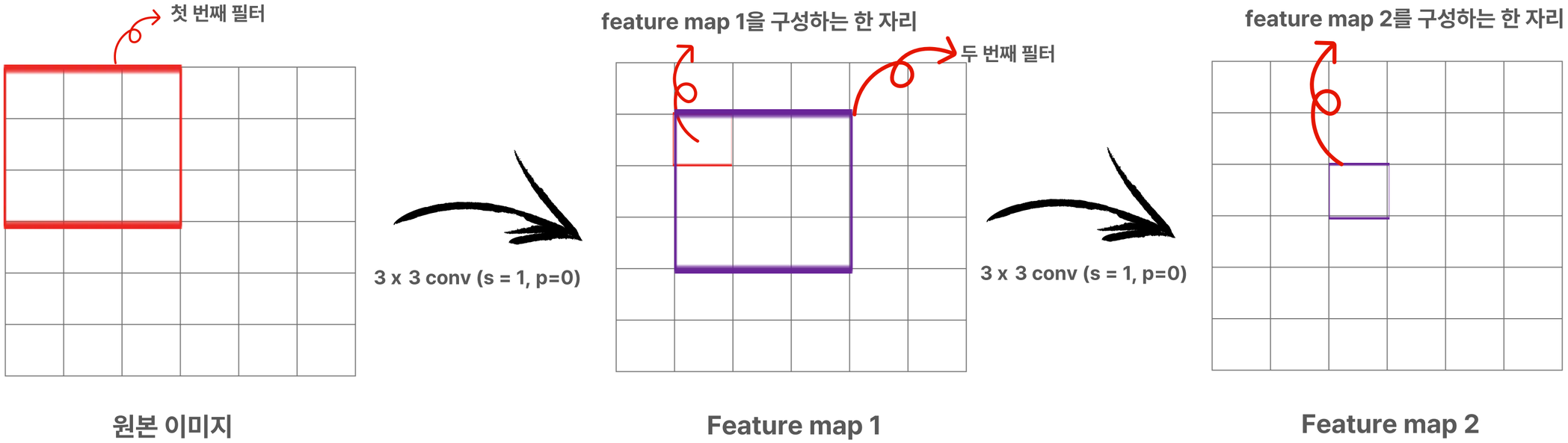
Pooling layer 역시 filter(학습 x)가 존재하고 stride만큼 움직이면서 해당 범위 내의 값들을 평균내거나 최댓값 선택

왜 사용하는가가 중요!

CNN: Pooling Layer

Receptive Field

Feature map을 구성하는 각 자리가 원본 이미지(or 데이터)의 어느 영역까지 정보를 담고 있는가?

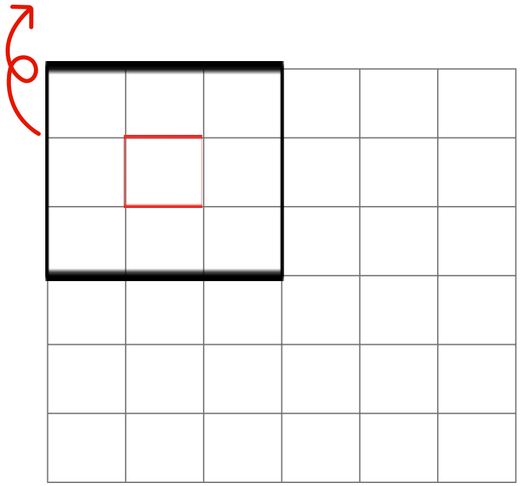


CNN: Pooling Layer

Receptive Field

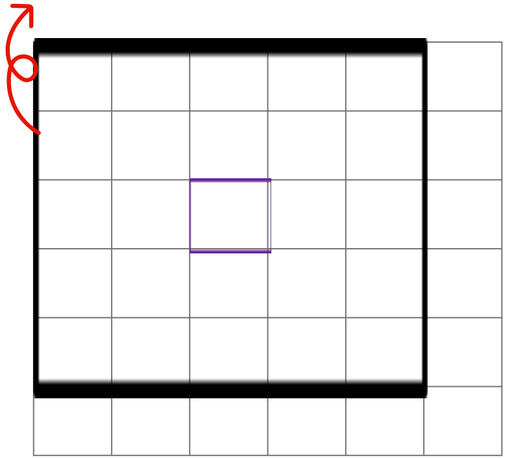
Feature map을 구성하는 각 자리가 원본 이미지(or 데이터)의 어느 영역까지 정보를 담고 있는가?

feature map 1을 구성하는 한 자리가 대표하는 원본 이미지 영역



Feature map 1

feature map 2를 구성하는 한 자리가 대표하는 원본 이미지 영역



Feature map 2

Conv을 진행하면 할수록 feature map 픽셀 하나가 대표하는 원본 이미지 영역이 증가함 = CNN에서 층을 거치면 거칠수록 feature map에 global context가 많이 담김

따라서 receptive field를 증가시킬수록 모델이 데이터의 global context를 잘 이해하게 되는데, 보통 RF를 증가시키는 방법이 stride를 늘려 띄엄띄엄 읽거나 pooling으로 과감하게 픽셀들을 제거할 때라 이 경우에는 모델의 local context or detail 이해력을 떨어뜨릴 수 있어 trade-off가 존재

CNN: Pooling Layer

Pooling Layer의 존재 이유

1. Receptive field를 늘려 모델의 global context 이해력 증가 (but local context or detail 이해력은 줄어듦)

2. 계산해야하는 것들을 줄여주니깐, computation load 및 학습 파라미터 수를 급격하게 줄여줌 (overfitting 방지)

CNN: Pooling Layer 차원 설계

거듭 강조하지만, 딥러닝에서 차원 계산은 매우 중요!

Input tensor의 차원: W x H x D ex) 컬러(RGB) 이미지: 224 x 224 x 3

Pooling Filter의 차원: F x F x D x 1 (Pooling filter는 1개로 고정임)

stride = s

padding은 pooling layer에서 쓰지 않음

Output의 차원: W2 x H2 x D2 라고 하면,

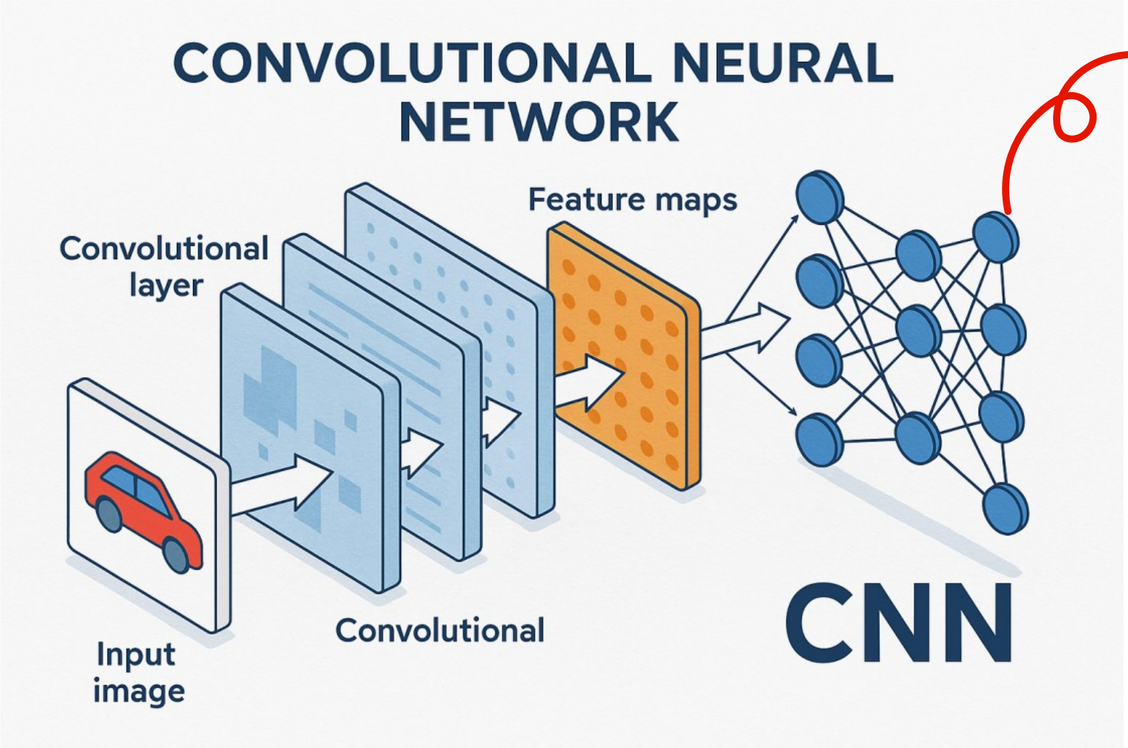
W2 = ((W - F)/s) + 1

H2 = ((H - F)/s) + 1

알아두면 좋은 식

D2 = D (pooling은 depth차원을 바꾸지 않음 = depth별 독립 연산)

CNN: Fully-Connected Layer (FC Layer)

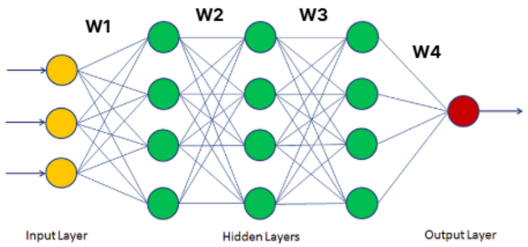


FC Layer

Multi-Layer Perceptron (MLP)

각 layer의 노드들이 모두 다 연결이 되어 있으면 fully-connect layer(FC layer)라고 부름

hidden layer는 모델이 인풋을 받고 중간에 나오는 값들을 의미 (은닉층 값이라고 부름)



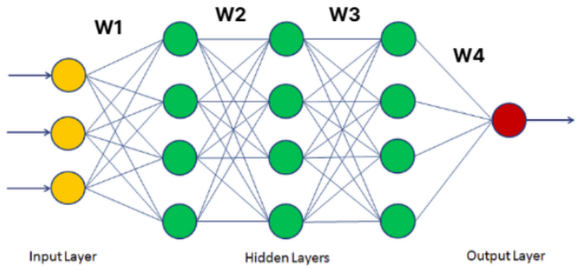
퍼셉트론을 여러 층 쌓아 만든 기초적인 딥러닝 모델

주로 정보가 혼합된 벡터를 모델에 이해시키고자 할 때 사용

Recap

CNN: Fully-Connected Layer (FC Layer)

Conv & Pooling x n → 데이터 내부 feature 추출 → 벡터화

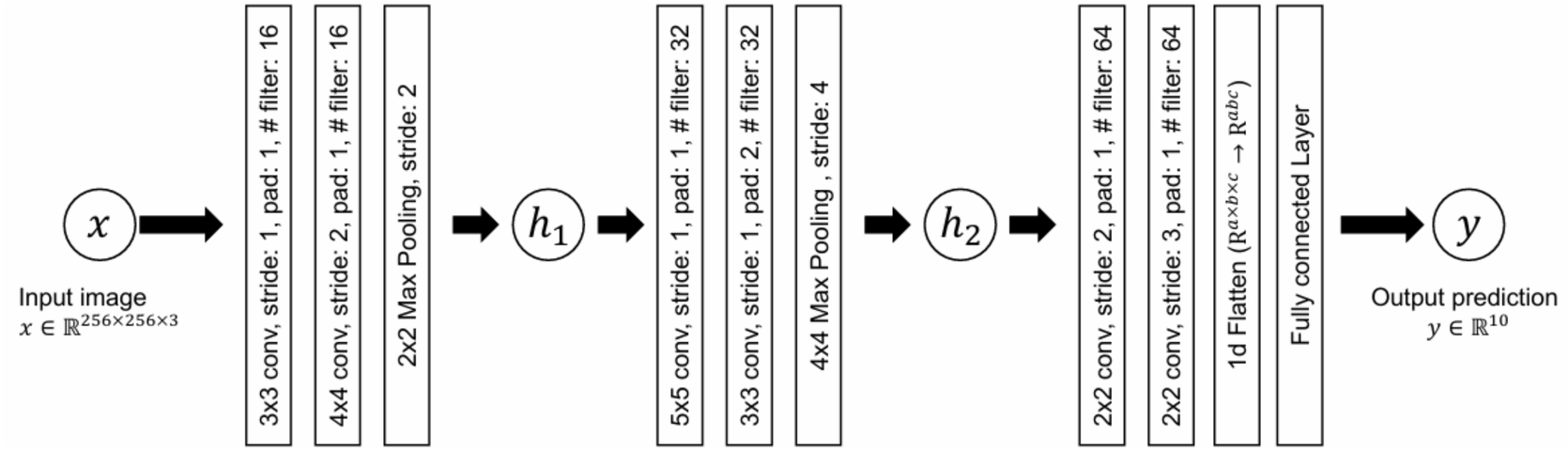


feature가 혼합되어 있는 벡터를 FC layer가 이해하여 task 수행

CNN: 차원 설계 연습

첨단컴퓨팅학부 전공수업 심층신경망 기말고사 문제

solution을 런어스에 제공을 해주셔서 가지고 와봤습니다! 이 문제를 풀 줄 아신다면 CNN이 어떤 구조를 가지는지 완벽하게 이해하셨다고 봐도 될 것 같아요



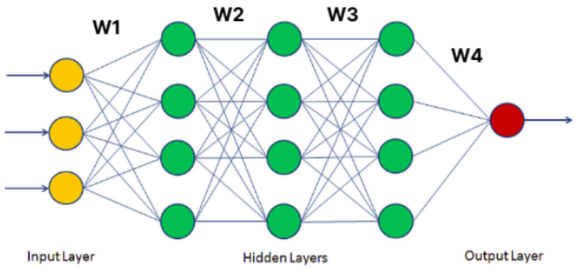
1. (5 points) What are dimensions of h_1, h_2?
   Answer: $h_1 : 64 \times 64 \times 16$, $h_2 : 16 \times 16 \times 32$

2. (5 points) Let assume the last fully connected layer is modeled with a single weight matrix $W \in \mathbb{R}^{d\times10}$. Then, what is the correct number for d?
   Answer: The dimension of hidden feature right before fully connected layer is: $4 \times 4 \times 64$. Therefore, $d = 1024$.

3. (5 points) How many number of parameters are introduced for this CNN?
   Answer: $(3\times3\times3+1)\times16+(4\times4\times16+1)\times16+(5\times5\times16+1)\times32+(3\times3\times32+1)\times32+(2\times2\times32+1)\times64+(2\times2\times64+1)\times64+1024\times10=61584$

Pretraining & Fine-tuning

Conv & Pooling x n → 데이터 내부 feature 추출 → 벡터화



사전학습 진행 → 모델 파라미터 가중치 파일 업로드

해당 가중치 파일을 로드해서 사용자는 다른 task에 대해 해당 모델의 일부 layer만 학습 시도 가능 = fine-tuning
(사전학습된 지식을 사용해 새로운 task에 대한 학습 진행)

새로 학습시키려는 task가 사전학습 task와 어느정도 alignment가 맞아야 함

2주차 Basic Deep Learning Summary

MLP, CNN은 나온지 오래됐고, 딥러닝의 기초 모델이지만 아직도 여전히 많이 쓰이기에 구조와 동작 원리를 이해하고 있어야 함

이 둘과 함께 설명한 딥러닝 기초 개념들(경사하강법, backpropagation, 차원 계산 등등) 역시 비단 MLP, CNN에만 쓰이는게 아닌 일반적인 딥러닝 모델들에 다 적용되는 것이기 때문에 매우 중요